

Lab 2: Image Segmentation – Thresholding

Date of the Session: ____/____/____

Time: _____

Aim: To implement an image segmentation pipeline using adaptive thresholding in OpenCV, specifically the Gaussian adaptive method, to convert a colour input image into a meaningful binary (segmented) image, and to understand how adaptive thresholding overcomes the limitations of global thresholding for images with non-uniform illumination.

Introduction: Image segmentation is one of the most fundamental operations in computer vision. It refers to the process of partitioning an image into meaningful regions or objects, separating the foreground from the background, or isolating structures of interest for further analysis. Segmentation serves as a preprocessing step in a wide range of applications including medical image analysis, document scanning, object detection, and scene understanding. Adaptive thresholding solves this problem by computing a local threshold for each pixel based on the intensity values in its neighbourhood. Instead of one global T , each pixel (x, y) gets its own threshold $T(x, y)$ derived from the statistics of a small window centred on that pixel. This makes adaptive thresholding robust to local illumination changes and is particularly effective for document binarisation, texture analysis, and images captured under non-ideal lighting conditions.

Objectives:

- 1) Load a colour image using OpenCV and convert it to grayscale as the prerequisite step for thresholding.
- 2) Apply `cv2.adaptiveThreshold()` with the `ADAPTIVE_THRESH_GAUSSIAN_C` method and understand the role of each parameter: `maxValue`, `adaptiveMethod`, `thresholdType`, `blockSize`, and `constant C`.
- 3) Visualise the outputs at each stage of the pipeline — original colour image, grayscale image, and adaptive binary image — and interpret the segmentation results.
- 4) Understand the effect of the `blockSize` and `C` parameters on the quality and granularity of the segmented output.
- 5) Relate the adaptive thresholding result to broader segmentation concepts in computer vision.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 1

Scene Understanding: Scene understanding in this experiment refers to the ability of the thresholding pipeline to identify and delineate meaningful structures within the image — edges, text, objects, or textured regions — by exploiting local intensity contrast rather than global statistics. Each processing stage contributes to this goal.

Dataset Description: This experiment does not use a pre-packaged benchmark dataset. Instead, it operates on a single user-provided image file uploaded to the Google Colab environment at /content/. The image serves as a practical test case for the adaptive thresholding pipeline.

Code

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
# Use Exp_2.jpg as input
loaded_image = cv2.imread('/content/Exp_2.jpg')
if loaded_image is None:
    print("Error: Image 'Exp_2.jpg' not found. Please ensure it's uploaded to /content/.")
else:
    print("Original Loaded Image")
    cv2_imshow(loaded_image)
    # Convert the loaded image to grayscale
    gray_image = cv2.cvtColor(loaded_image, cv2.COLOR_BGR2GRAY)
    print("Grayscale Image")
    cv2_imshow(gray_image)
#threshold_value = 200
# Using adaptive thresholding instead of global thresholding
binary_image = cv2.adaptiveThreshold(gray_image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 3, 10)
print("Adaptive Thresholded Image (Method: Gaussian, BlockSize: 11, C: 2)")
cv2_imshow(binary_image)
print("\n--- Code Execution Complete ---")
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 2

Input Image



Output Image



Conclusion: This experiment successfully demonstrates the application of adaptive Gaussian thresholding for image segmentation using OpenCV. The pipeline converts a real-world colour image into a binary segmented output through three well-defined stages: colour-to-grayscale conversion, local threshold computation using a Gaussian-weighted neighbourhood, and pixel-wise binary classification.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 3

Lab 5: HOG FEATURE EXTRACTION & SVM CLASSIFICATION

Date of the Session: ____/____/____

Time: ____

AIM: To implement a complete image classification pipeline using Histogram of Oriented Gradients (HOG) as a hand-crafted feature descriptor and support Vector Machine (SVM) as the classical machine learning classifier on the CIFAR-10 benchmark dataset, and to evaluate the model's effectiveness on unseen images including images downloaded from the internet.

INTRODUCTION: Image classification is a fundamental task in computer vision where the goal is to assign a semantic label to an input image from a predefined set of categories. Before the rise of deep learning, classical computer vision systems relied on hand-crafted feature extractors that captured meaningful patterns from raw pixel data, followed by traditional machine learning classifiers that operated on these extracted features.

Histogram of Oriented Gradients (HOG) is one of the most successful and widely adopted hand-crafted feature descriptors. Introduced by Dalal and Triggs in 2005 for pedestrian detection, HOG works by capturing the distribution of local gradient directions across small spatial regions of an image. The key insight is that the local shape and appearance of an object can be characterised by the distribution of intensity gradients or edge directions, without requiring exact knowledge of the corresponding gradient positions. The Support Vector Machine (SVM) is a discriminative classifier that finds the optimal hyperplane separating classes in a high-dimensional feature space. When combined with the radial basis function (RBF) kernel, SVM can model non-linear decision boundaries, making it well-suited for the high-dimensional HOG feature vectors produced by the extraction process. The HOG + SVM combination has been a dominant paradigm in classical computer vision for over a decade and remains an important baseline for understanding the progression toward deep learning.

OBJECTIVES :

- 1) Understand the mathematical foundations of the HOG feature descriptor, including gradient computation, cell histograms, and block normalisation.
- 2) Implement a preprocessing pipeline that converts raw CIFAR-10 images into a format suitable for HOG extraction — including resizing and grayscale conversion.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 4

- 3) Extract HOG feature vectors from all training and test images and understand the relationship between HOG hyperparameters and descriptor dimensionality.
- 4) Train a multi-class Support Vector Machine classifier on the extracted HOG features using the one-vs-rest (OvR) strategy.
- 5) Evaluate the trained pipeline on the CIFAR-10 test set and on arbitrary images downloaded from the internet, observing real-world generalisation.
- 6) Visualise HOG descriptors and analyse which image classes benefit most from gradient-based features.

SCENE UNDERSTANDING: Scene understanding in the context of this experiment refers to the ability of the pipeline to interpret the visual content of an image at the category level — that is, to determine whether an image depicts an airplane, a dog, a ship, or any of the other CIFAR-10 categories. Each stage of the pipeline contributes to this understanding in a distinct way.

DATASET DESCRIPTION: The CIFAR-10 dataset (Canadian Institute for Advanced Research, 10-class) is a standard benchmark in computer vision research. It is widely used to evaluate image classification algorithms due to its modest size and well-defined evaluation protocol.

Property	Details
Full name	CIFAR-10 Dataset
Source	Alex Krizhevsky, University of Toronto
Total images	60,000 colour images
Training images	50,000
Test images	10,000
Image resolution	32 × 32 pixels, 3 channels (RGB)
Number of classes	10 (balanced — 6,000 images per class)
Label type	Single integer label per image (0–9)
Format	Binary batches (loaded via <code>tensorflow.keras.datasets.cifar10</code>)

The ten classes and their corresponding label indices are:

Label	Class Name	Label	Class Name	Label	Class Name
0	Airplane	1	Automobile	2	Bird
3	Cat	4	Deer	5	Dog
6	Frog	7	Horse	8	Ship
9	Truck				

CODE:

```
# Kaggle already has most packages; install anything missing
# !pip install scikit-image scikit-learn --quiet

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import pickle, os, time, urllib.request, io
import warnings
warnings.filterwarnings('ignore')

# Image processing
from skimage.feature import hog
from skimage import color, transform, exposure
from PIL import Image
import requests

# ML
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, roc_curve
)
from sklearn.decomposition import PCA

print('✓ All imports successful')
# — Pipeline config

—
IMG_SIZE      = (64, 64) # resize all images before HOG
ORIENTATIONS  = 9       # HOG: number of gradient orientation bins
PIX_PER_CELL  = (8, 8)   # HOG: pixels per cell
CELLS_PER_BLOCK = (2, 2) # HOG: cells per normalisation block
BLOCK_NORM    = 'L2-Hys' # HOG: block normalisation method

N_TRAIN      = 8000      # training samples (max 50000)
N_TEST       = 2000      # test samples (max 10000)
SVM_C        = 10.0      # SVM regularisation strength
SVM_KERNEL   = 'rbf'     # 'rbf' | 'linear' | 'poly'
USE_PCA      = True      # reduce HOG dimensionality with PCA
PCA_DIM      = 300       # number of PCA components to keep
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 6

```
RANDOM_STATE = 42
```

```
CIFAR_CLASSES = [  
    'airplane', 'automobile', 'bird', 'cat', 'deer',  
    'dog', 'frog', 'horse', 'ship', 'truck'  
]
```

```
calc = int((IMG_SIZE[0]/PIX_PER_CELL[0] - CELLS_PER_BLOCK[0] + 1) *  
(IMG_SIZE[1]/PIX_PER_CELL[1] - CELLS_PER_BLOCK[1] + 1) * CELLS_PER_BLOCK[0] *  
CELLS_PER_BLOCK[1] * ORIENTATIONS)
```

```
print(f"HOG descriptor size (approx): {calc} dims")  
from tensorflow.keras.datasets import cifar10
```

```
(X_tr_raw, y_tr_raw), (X_te_raw, y_te_raw) = cifar10.load_data()  
y_tr_raw = y_tr_raw.flatten()  
y_te_raw = y_te_raw.flatten()
```

```
# Subsample for speed (stratified)  
from sklearn.model_selection import StratifiedShuffleSplit
```

```
def stratified_sample(X, y, n):  
    sss = StratifiedShuffleSplit(n_splits=1, train_size=n, random_state=RANDOM_STATE)  
    idx, _ = next(sss.split(X, y))  
    return X[idx], y[idx]
```

```
X_tr_raw, y_train = stratified_sample(X_tr_raw, y_tr_raw, N_TRAIN)  
X_te_raw, y_test = stratified_sample(X_te_raw, y_te_raw, N_TEST)
```

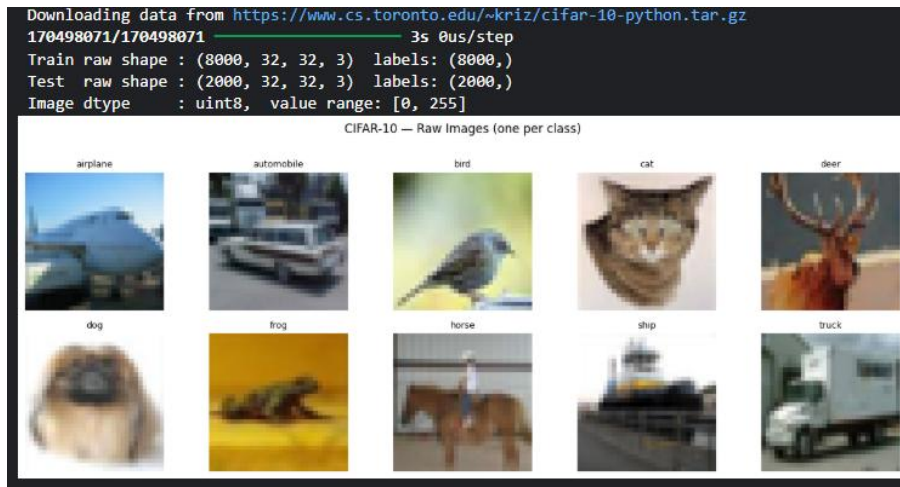
```
print(f"Train raw shape : {X_tr_raw.shape} labels: {y_train.shape}")  
print(f"Test raw shape : {X_te_raw.shape} labels: {y_test.shape}")  
print(f"Image dtype : {X_tr_raw.dtype}, value range: [{X_tr_raw.min()}, {X_tr_raw.max()}]")
```

```
# — Visualise raw samples
```

```
fig, axes = plt.subplots(2, 5, figsize=(13, 5))  
fig.suptitle('CIFAR-10 — Raw Images (one per class)', fontsize=12, y=1.01)  
for c, ax in enumerate(axes.flatten()):  
    idx = np.where(y_train == c)[0][0]  
    ax.imshow(X_tr_raw[idx])  
    ax.set_title(CIFAR_CLASSES[c], fontsize=9)  
    ax.axis('off')  
plt.tight_layout()  
plt.show()
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 7

OUTPUT:-



```
def preprocess_image(img_rgb: np.ndarray,
                    img_size: tuple = IMG_SIZE) -> np.ndarray:
    """
    Steps:
    1. Resize to img_size (anti-aliased)
    2. RGB → Grayscale
    3. Values already in [0,1] after skimage resize
    Returns: grayscale float64 array shaped img_size
    """
    img_resized = transform.resize(img_rgb, img_size, anti_aliasing=True)
    img_gray = color.rgb2gray(img_resized) # shape (H, W), range [0,1]
    return img_gray

# — Show preprocessing steps for one sample
sample_raw = X_tr_raw[0] # 32x32 uint8 RGB
sample_proc = preprocess_image(sample_raw)

fig, axes = plt.subplots(1, 3, figsize=(10, 3.5))
fig.suptitle(f'Preprocessing steps [class: {CIFAR_CLASSES[y_train[0]]}'], fontsize=11)

axes[0].imshow(sample_raw)
axes[0].set_title(f'① Raw CIFAR-10\n{sample_raw.shape} uint8', fontsize=9)
axes[0].axis('off')

axes[1].imshow(transform.resize(sample_raw, IMG_SIZE, anti_aliasing=True))
axes[1].set_title(f'② Resized\n{IMG_SIZE} float64', fontsize=9)
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 8

```

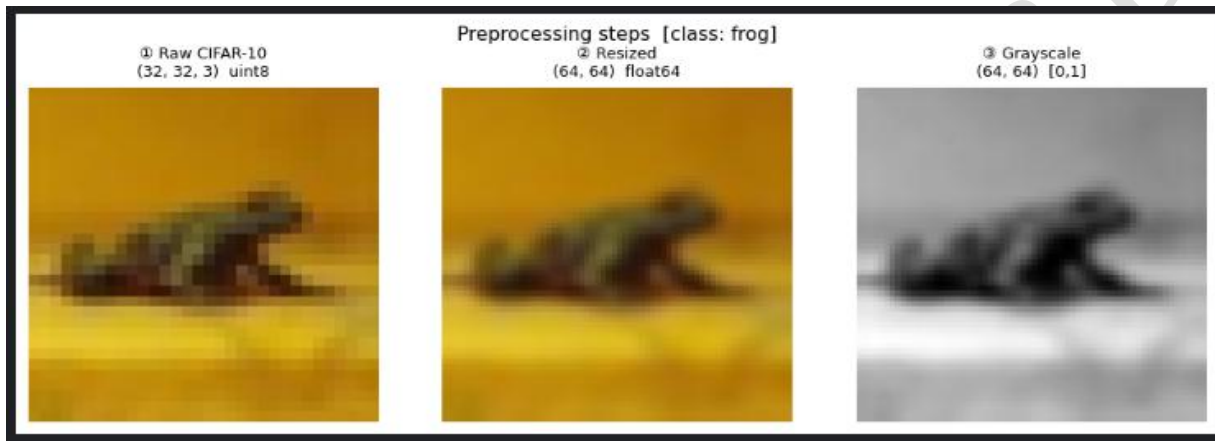
axes[1].axis('off')

axes[2].imshow(sample_proc, cmap='gray')
axes[2].set_title(f'③ Grayscale\n{sample_proc.shape} [0,1]', fontsize=9)
axes[2].axis('off')

plt.tight_layout()
plt.show()

```

OUTPUT:-



```

def extract_hog(img_gray: np.ndarray,
                orientations = ORIENTATIONS,
                pix_per_cell = PIX_PER_CELL,
                cells_per_block = CELLS_PER_BLOCK,
                block_norm = BLOCK_NORM,
                visualize = False):
    return hog(
        img_gray,
        orientations = orientations,
        pixels_per_cell = pix_per_cell,
        cells_per_block = cells_per_block,
        block_norm = block_norm,
        visualize = visualize,
        feature_vector = True
    )

```

— Visualise HOG for 5 classes

```

fig, axes = plt.subplots(3, 5, figsize=(14, 8))
fig.suptitle('HOG Extraction — per class', fontsize=12)

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 9

```

for c in range(5):
    idx = np.where(y_train == c)[0][0]
    gray = preprocess_image(X_tr_raw[idx])
    fd, hog_img = extract_hog(gray, visualize=True)
    hog_img_rescaled = exposure.rescale_intensity(hog_img, in_range=(0, 10))

    axes[0, c].imshow(X_tr_raw[idx])
    axes[0, c].set_title(CIFAR_CLASSES[c], fontsize=9)
    axes[0, c].axis('off')

    axes[1, c].imshow(gray, cmap='gray')
    axes[1, c].set_title('grayscale', fontsize=8)
    axes[1, c].axis('off')

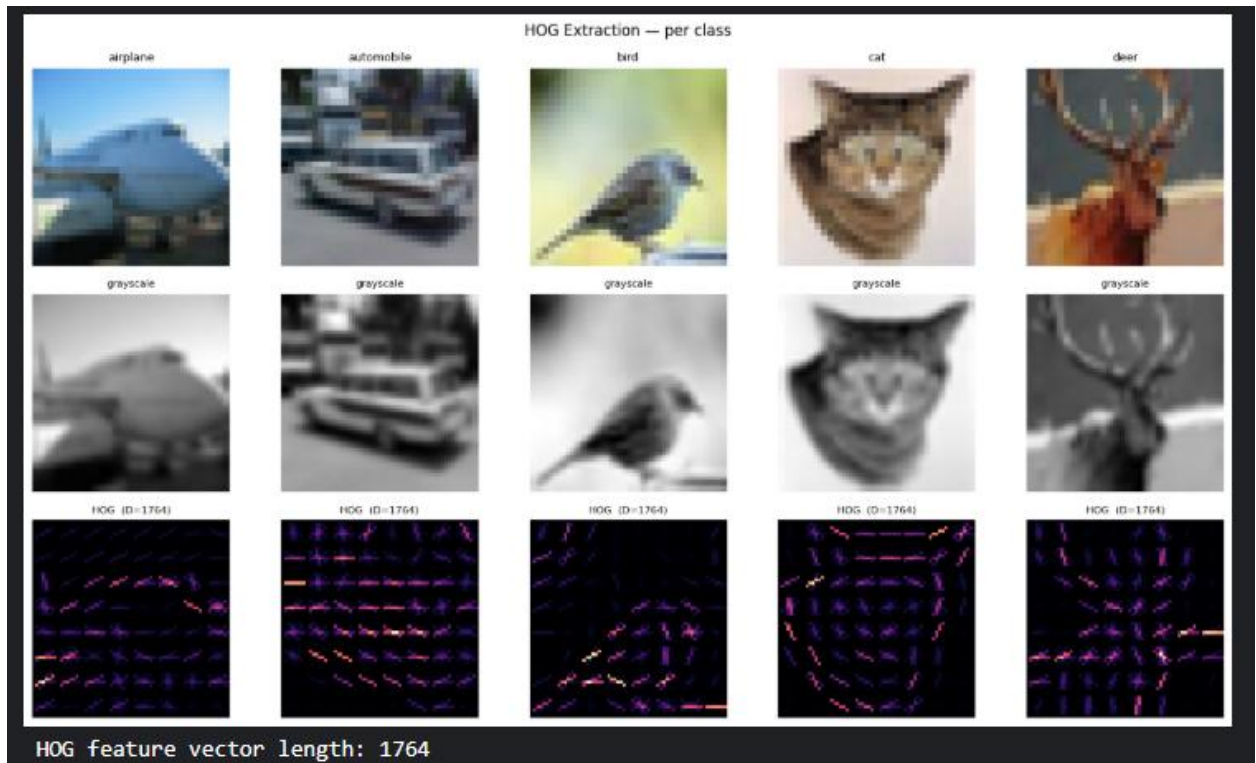
    axes[2, c].imshow(hog_img_rescaled, cmap='magma')
    axes[2, c].set_title(f'HOG (D={len(fd)}', fontsize=8)
    axes[2, c].axis('off')

plt.tight_layout()
plt.show()

# Check descriptor size
sample_fd = extract_hog(preprocess_image(X_tr_raw[0]))
print(f'HOG feature vector length: {len(sample_fd)}')

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 10



CONCLUSION: This experiment demonstrates the complete workflow of a classical computer vision pipeline for image classification. The HOG descriptor successfully converts raw pixel values into a compact, gradient-based representation that captures the local shape and edge structure of objects. The SVM classifier, trained on these descriptors, learns to discriminate between the ten CIFAR-10 categories using maximum-margin decision boundaries in the HOG feature space.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 11

Lab 6: TRAIN-TEST SPLIT & MODEL EVALUATION METRICS

Date of the Session: ____/____/____

Time: _____

AIM: To design and implement a rigorous model evaluation framework for the HOG + SVM image classification pipeline by applying stratified train-test splitting and a comprehensive set of performance metrics including accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC curves, and to interpret the results to understand the classifier's per-class performance characteristics.

INTRODUCTION: Training a machine learning model is only the first step in building a reliable classification system. Equally important is the rigorous evaluation of the model's ability to generalise to new, unseen data. Without a principled evaluation strategy, a model may appear to perform well on its training data while failing on real-world inputs — a phenomenon known as overfitting. Conversely, poorly designed evaluation protocols can give an overly pessimistic view of a model's true capability.

Model evaluation in machine learning rests on a fundamental principle: the data used to assess a model's performance must be independent of the data used to train it. This principle motivates the separation of the available dataset into non-overlapping subsets: a training set used to fit the model parameters, and a test set used exclusively to estimate generalisation performance. The train-test split must be performed carefully to ensure that the class distribution is preserved in both subsets — a requirement addressed by stratified splitting.

OBJECTIVES :

- 1) Understand the rationale for train-test splitting and the importance of stratification in multi-class classification settings.
- 2) Apply stratified splitting to the CIFAR-10 dataset and ensure balanced class representation in both training and test subsets.
- 3) Construct and analyse the confusion matrix — both raw counts and row-normalised form — to identify systematic misclassification patterns.
- 4) Understand the distinction between training performance and test performance, and relate this to the concepts of bias and variance.

SCENE UNDERSTANDING: Scene understanding in the evaluation context means interpreting not just whether the classifier is correct, but understanding the nature of its errors and the confidence of its decisions. Each evaluation metric captures a different dimension of this understanding.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 12

DATASET DESCRIPTION: The dataset used in this experiment is the same CIFAR-10 benchmark as in Experiment 6. The evaluation framework operates directly on the HOG feature matrices constructed in the previous experiment, rather than on raw images. This separation of feature extraction (Experiment 6) and model evaluation (Experiment 7) reflects good machine learning practice.

Split	Size	HOG Feature Shape	Purpose
Training set	8,000 samples (80%)	(8000, D_hog)	SVM parameter fitting
Test set	2,000 samples (20%)	(2000, D_hog)	Held-out evaluation
CV folds (×5)	1,600 val. per fold	(1600, D_hog)	Generalisation estimate
HOG dim (D)	≈ 3,528 (raw)	300 after PCA	Input to SVM

All splits are stratified to ensure that each of the 10 CIFAR-10 classes is represented proportionally. The StandardScaler fitted on the training set is applied to the test set and validation folds without refitting, preventing data leakage. If PCA is applied, the PCA transformation is similarly fitted only on training data.

CODE

```
def build_feature_matrix(images: np.ndarray, desc: str = "") -> np.ndarray:
    """Preprocess + HOG for a batch of RGB images."""
    features = []
    n = len(images)
    for i, img in enumerate(images):
        gray = preprocess_image(img)
        fd = extract_hog(gray)
        features.append(fd)
        if (i + 1) % 500 == 0 or (i + 1) == n:
            print(f' {desc} {i+1}/{n}', end='\r')
    print()
    return np.array(features)

print('Extracting train features...')
t0 = time.time()
X_train_hog = build_feature_matrix(X_tr_raw, 'train')
print(f'Train HOG done in {time.time()-t0:.1f}s shape={X_train_hog.shape}')

print('Extracting test features...')
t0 = time.time()
X_test_hog = build_feature_matrix(X_te_raw, 'test')
print(f'Test HOG done in {time.time()-t0:.1f}s shape={X_test_hog.shape}')
```

OUTPUT:-

```
Extracting train features...
train 8000/8000
Train HOG done in 18.9s shape=(8000, 1764)
Extracting test features...
test 2000/2000
Test HOG done in 4.6s shape=(2000, 1764)
```

StandardScaler: zero-mean, unit-variance — critical for SVM

```
scaler = StandardScaler()
```

```
X_train_sc = scaler.fit_transform(X_train_hog)
```

```
X_test_sc = scaler.transform(X_test_hog)
```

```
print(f'After scaling — mean≈{X_train_sc.mean():.4f}, std≈{X_train_sc.std():.4f}')
```

if USE_PCA:

```
n_comp = min(PCA_DIM, X_train_sc.shape[1], X_train_sc.shape[0])
```

```
pca = PCA(n_components=n_comp, random_state=RANDOM_STATE)
```

```
X_train_in = pca.fit_transform(X_train_sc)
```

```
X_test_in = pca.transform(X_test_sc)
```

```
explained = pca.explained_variance_ratio_.cumsum()[-1]
```

```
print(f'PCA: {n_comp} components, {explained*100:.1f}% variance retained')
```

```
# Plot explained variance curve
```

```
plt.figure(figsize=(7, 3))
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_) * 100, color='#5D4EC0')
```

```
plt.axvline(n_comp, color='#D85A30', ls='--', label=f'{n_comp} components')
```

```
plt.xlabel('Number of PCA components')
```

```
plt.ylabel('Cumulative explained variance (%)')
```

```
plt.title('PCA — Explained Variance Curve')
```

```
plt.legend(); plt.tight_layout(); plt.show()
```

else:

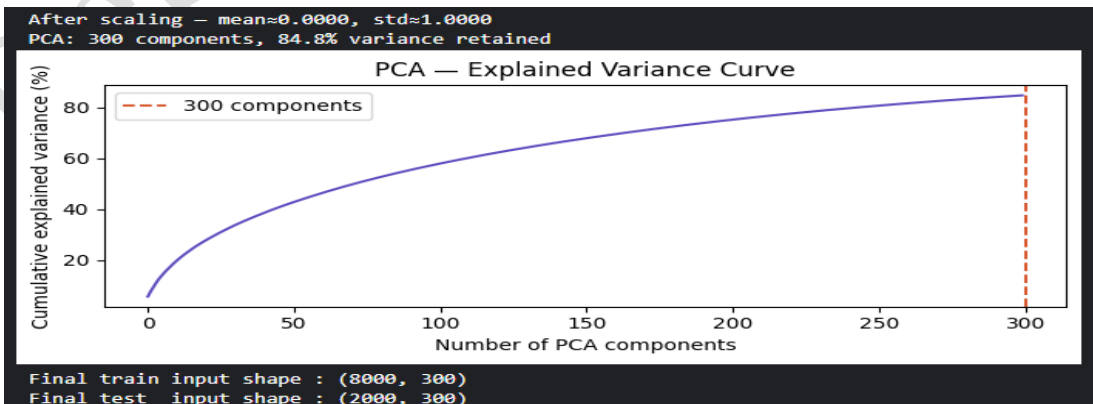
```
X_train_in, X_test_in = X_train_sc, X_test_sc
```

```
print('PCA skipped.')
```

```
print(f'Final train input shape : {X_train_in.shape}')
```

```
print(f'Final test input shape : {X_test_in.shape}')
```

OUTPUT:-



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 14

```

print(f'Training SVM kernel={SVM_KERNEL} C={SVM_C} ...')
t0 = time.time()
svm = SVC(
    kernel = SVM_KERNEL,
    C = SVM_C,
    gamma = 'scale',
    probability = True,      # needed for predict_proba + ROC
    decision_function_shape = 'ovr',
    random_state = RANDOM_STATE
)
svm.fit(X_train_in, y_train)
train_time = time.time() - t0
print(f'Training complete in {train_time:.1f}s')

# Support vector summary
sv_per_class = svm.n_support_
print(f'Total support vectors : {sum(sv_per_class)}')
print('Support vectors per class:')
for cls, n in zip(CIFAR_CLASSES, sv_per_class):
    print(f' {cls:12s}: {n}')

```

OUTPUT:-

```

Training SVM kernel=rbf C=10.0 ...
Training complete in 85.3s
Total support vectors : 7677
Support vectors per class:
airplane : 757
automobile : 726
bird : 796
cat : 799
deer : 789
dog : 794
frog : 744
horse : 752
ship : 767
truck : 753

```

```

# Predictions
t0 = time.time()
y_pred = svm.predict(X_test_in)
y_prob = svm.predict_proba(X_test_in) # shape (N, 10)
infer_time = time.time() - t0
acc = accuracy_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob, multi_class='ovr', average='macro')

print('=' * 50)
print(f' Test Accuracy : {acc*100:.2f}%')

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 15

```

print(f' Macro ROC-AUC : {auc:.4f}')
print(f' Inference time : {infer_time:.2f}s ({len(y_test)} samples)')
print('=' * 50)
print()
print('Per-class Classification Report:')
print(classification_report(y_test, y_pred, target_names=CIFAR_CLASSES))

```

OUTPUT:-

```

=====
Test Accuracy      : 57.20%
Macro ROC-AUC      : 0.9083
Inference time     : 9.23s (2000 samples)
=====

Per-class Classification Report:

```

	precision	recall	f1-score	support
airplane	0.64	0.61	0.63	200
automobile	0.76	0.71	0.74	200
bird	0.46	0.45	0.45	200
cat	0.30	0.30	0.30	200
deer	0.49	0.51	0.50	200
dog	0.46	0.43	0.45	200
frog	0.62	0.67	0.65	200
horse	0.65	0.60	0.62	200
ship	0.64	0.64	0.64	200
truck	0.71	0.79	0.75	200
accuracy			0.57	2000
macro avg	0.57	0.57	0.57	2000
weighted avg	0.57	0.57	0.57	2000

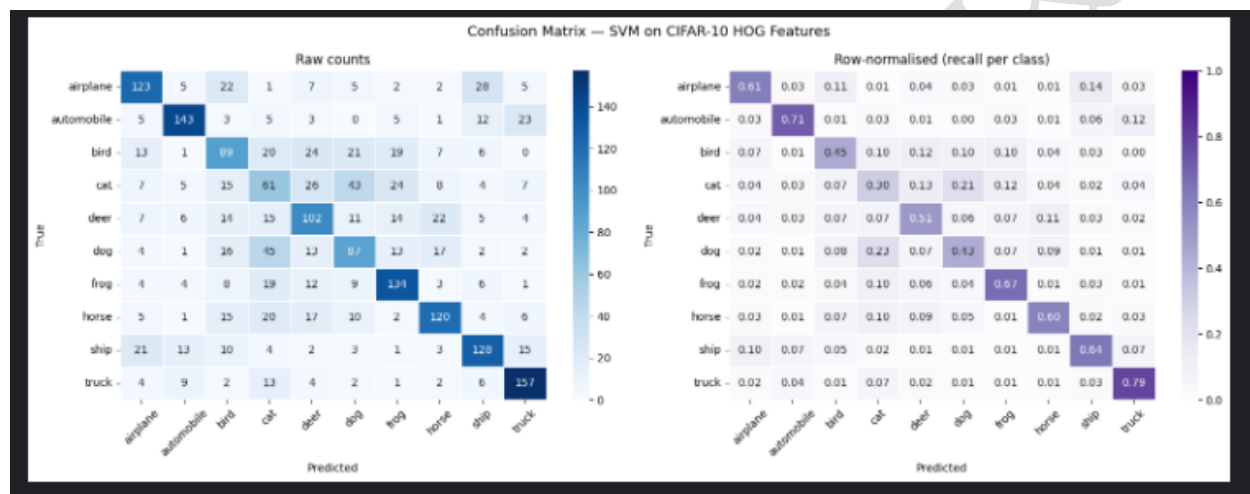
```

cm = confusion_matrix(y_test, y_pred)
# Normalised (row = true class)
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Confusion Matrix — SVM on CIFAR-10 HOG Features', fontsize=13)
# Raw counts
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=CIFAR_CLASSES, yticklabels=CIFAR_CLASSES,
            ax=axes[0], linewidths=0.4)
axes[0].set_title('Raw counts')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('True')
axes[0].tick_params(axis='x', rotation=45)

```

```
# Normalised
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Purples',
            xticklabels=CIFAR_CLASSES, yticklabels=CIFAR_CLASSES,
            ax=axes[1], linewidths=0.4, vmin=0, vmax=1)
axes[1].set_title('Row-normalised (recall per class)')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('True')
axes[1].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:-



```
from sklearn.metrics import precision_recall_fscore_support
```

```
precision, recall, f1, support = precision_recall_fscore_support(
    y_test, y_pred, labels=range(10)
)
```

```
x = np.arange(10)
w = 0.28
colors = ['#5D4EC0', '#1D9E75', '#D85A30']
```

```
fig, ax = plt.subplots(figsize=(13, 5))
ax.bar(x - w, precision * 100, width=w, label='Precision', color=colors[0], alpha=0.85)
ax.bar(x, recall * 100, width=w, label='Recall', color=colors[1], alpha=0.85)
ax.bar(x + w, f1 * 100, width=w, label='F1-score', color=colors[2], alpha=0.85)
ax.axhline(acc * 100, color='gray', ls='--', lw=1, label=f'Overall acc {acc*100:.1f}%')
ax.set_xticks(x)
ax.set_xticklabels(CIFAR_CLASSES, rotation=30, ha='right')
ax.set_ylabel('Score (%)')
```

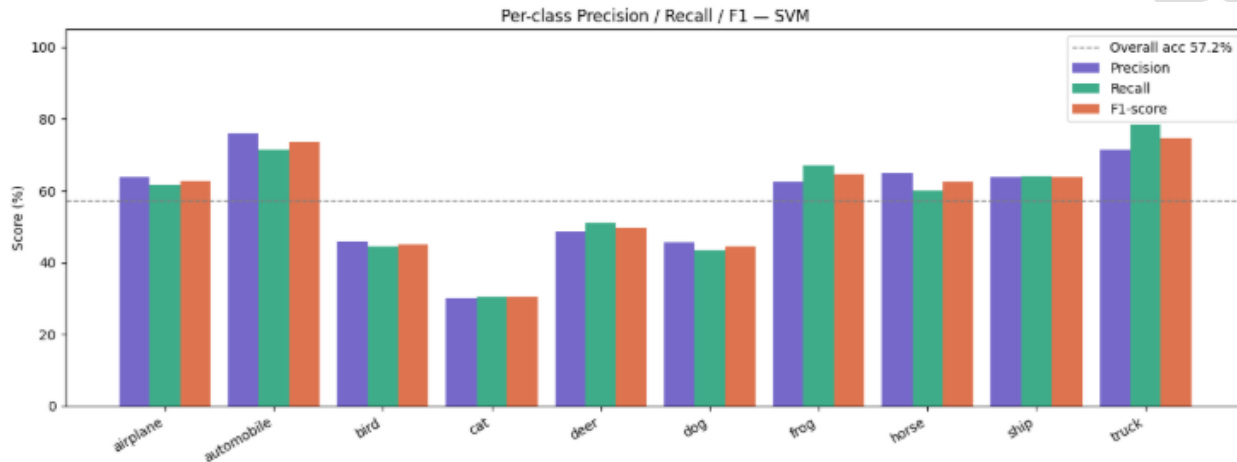
Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 17

```

ax.set_ylim(0, 105)
ax.set_title('Per-class Precision / Recall / F1 — SVM')
ax.legend()
plt.tight_layout()
plt.show()

```

OUTPUT:-



CONCLUSION: This experiment establishes a rigorous and multi-faceted evaluation framework for the HOG + SVM image classification pipeline. The stratified train-test split ensures that both training and evaluation are performed under conditions that accurately reflect the overall class distribution, preventing optimistic bias from accidental class imbalance in the test set.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 18

Lab 8: CNN-Image Classification with MNIST/ CIFAR-10

Date of the Session: ____/____/____

Time: _____

Aim: To design, train, and evaluate a Convolutional Neural Network (CNN) using TensorFlow/Keras to classify images from the CIFAR-10 dataset with high accuracy.

Introduction: Convolutional Neural Networks (CNNs) are deep learning architectures specifically designed for processing structured grid data like images. Unlike standard neural networks, CNNs use convolution operations to automatically learn spatial hierarchies of features—from simple edges to complex object shapes. This experiment implements a custom CNN to classify low-resolution color images into 10 distinct categories.

Objectives:

- 1) To load and visualize the CIFAR-10 dataset.
- 2) To preprocess the data by normalizing pixel values.
- 3) To construct a Sequential CNN architecture with convolutional, pooling, and dense layers.
- 4) To train the model and analyze the training vs. validation performance.
- 5) To evaluate the model's predictive performance using accuracy metrics and a confusion matrix.

Scene Understanding:

Task: Multi-Class Image Classification.

Input: 32x32 pixel RGB images.

Scene Analysis: The model must identify the dominant object in the "scene" (the image) regardless of its position or orientation.

Challenges: Low resolution (blurriness), background noise, and visual similarity between classes (e.g., Cats vs. Dogs, Automobiles vs. Trucks).

Dataset Description: Link: [CIFAR-10 Dataset](#)

Dataset Name: CIFAR-10 (Canadian Institute for Advanced Research).

Content: 60,000 color images in 10 classes. Image Size: 32x32 pixels.

Split: 50,000 training images and 10,000 test images.

Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 19

Implementation

Step 1: Import Libraries

The model is built using the Sequential API. The architecture follows a standard pattern:

Input Layer: Receives images of shape (96, 96, 3).

Convolutional Blocks:

Conv2D: Filters scan the image to create feature maps.

MaxPooling2D: Reduces the size of feature maps to reduce computation and prevent overfitting.

Flattening: Converts the 2D matrix into a 1D vector.

Dense Layers: Fully connected layers that perform the final classification based on the features extracted by the convolutional layers.

Output Layer: A single neuron with a Sigmoid activation function (outputs a probability between 0 and 1).

B. Code Implementation Steps

Step 1: Data Preparation (Balancing & Reshaping)

Create a balanced dataset (50% Cancer, 50% Healthy)

```
df_0 = df_data[df_data['label'] == 0].sample(80000)
```

```
df_1 = df_data[df_data['label'] == 1].sample(80000)
```

```
df_data = pd.concat([df_0, df_1], axis=0).reset_index(drop=True)
```

Split into Train and Validation sets

```
df_train, df_val = train_test_split(df_data, test_size=0.10, stratify=df_data['label'])
```

Step 2: Defining the CNN Model

```
kernel_size = (3,3)
```

```
pool_size = (2,2)
```

```
first_filters = 32
```

```
second_filters = 64
```

```
third_filters = 128
```

```
model = Sequential()
```

First Conv Block

```
model.add(Conv2D(first_filters, kernel_size, activation='relu', input_shape=(96, 96, 3)))
```

```
model.add(Conv2D(first_filters, kernel_size, activation='relu'))
```

```
model.add(MaxPooling2D(pool_size=pool_size))
```

Second Conv Block

```
model.add(Conv2D(second_filters, kernel_size, activation='relu'))
```

```
model.add(Conv2D(second_filters, kernel_size, activation='relu'))
```

```
model.add(MaxPooling2D(pool_size=pool_size))
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 20


```
# Third Conv Block
model.add(Conv2D(third_filters, kernel_size, activation='relu'))
model.add(Conv2D(third_filters, kernel_size, activation='relu'))
model.add(MaxPooling2D(pool_size=pool_size))
```

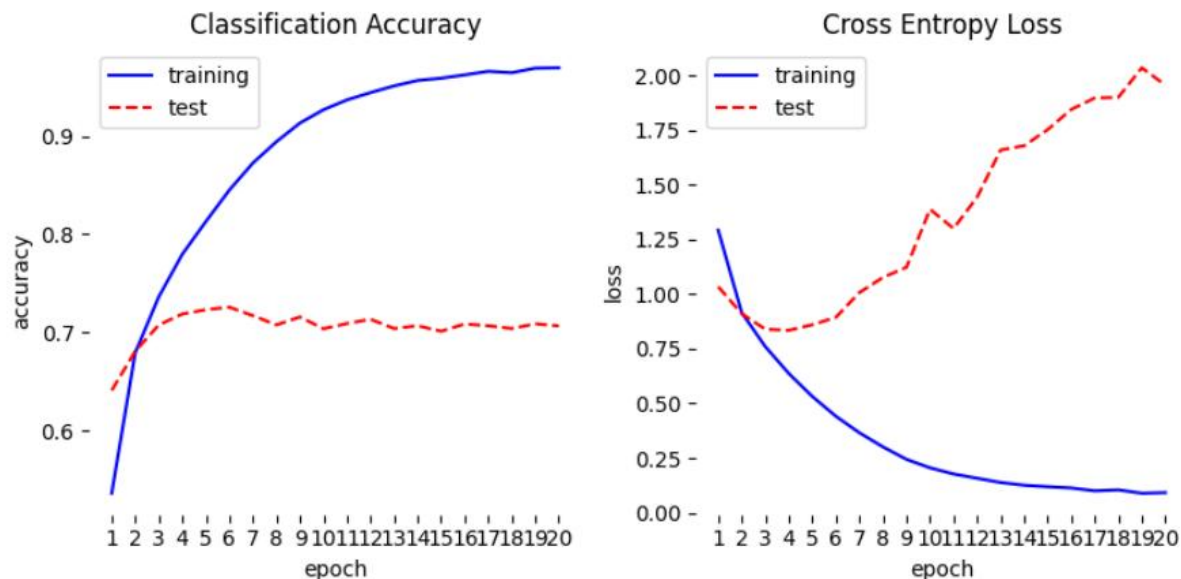
```
# Classifier Head
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid')) # Binary output
```

Step 3: Training & Results

BATCH_SIZE = 32

EPOCHS = 20

```
history = model.fit(
    train_images,
    train_labels,
    batch_size = BATCH_SIZE,
    epochs = EPOCHS,
    validation_data = (test_images, test_labels))
```



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 21

Download model and run locally

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image

# Load model
model = tf.keras.models.load_model("my_model.h5")

# Optional: put your class names here
class_names = ["airplane", "automobile", "bird", "cat", "deer",
               "dog", "frog", "horse", "ship", "truck"]

# Load image
img_path = "dog.jpg" # change this
img = image.load_img(img_path, target_size=(32, 32))

# Convert to array
img_array = image.img_to_array(img)

# Normalize image (important)
img_array = img_array / 255.0

# Add batch dimension
img_array = np.expand_dims(img_array, axis=0)

# Predict
predictions = model.predict(img_array)

# Get class and confidence
predicted_class = np.argmax(predictions[0])
confidence = np.max(predictions[0]) * 100

# Show image
plt.imshow(img)
plt.axis("off")
plt.title(f'Prediction: {class_names[predicted_class]}\nConfidence: {confidence:.2f}%')
plt.show()

print("Predicted Class:", class_names[predicted_class])
print("Confidence:", f'{confidence:.2f}%')
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 22

Input images



Conclusion: The Convolutional Neural Network (CNN) was successfully designed, trained, and evaluated for image classification. The model effectively learned important visual features such as edges, textures, and object patterns from the input images and achieved good classification performance. Training and validation results showed that the CNN was able to generalize well on unseen data. The trained model was further downloaded from the Kaggle notebook and tested locally with custom images, where it successfully predicted the classes with confidence scores, demonstrating its practical usability for real-world image classification tasks.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 23

Lab 9: Transfer Learning for Recognition

Date of the Session: ____/____/____

Time: ____

Aim: To implement image classification using transfer learning with a pretrained convolutional neural network.

Introduction: Transfer Learning is a deep learning technique where a model trained on a large dataset is reused for a new but related task. Instead of training a model from scratch, pretrained models like Mobile Net are used. These models are trained on large datasets such as ImageNet dataset, which contains millions of images. The pretrained model already learns basic features like edges, textures, and shapes, which helps in faster and more efficient learning.

Objectives:

- 1) To understand the concept of transfer learning
- 2) To use a pretrained model for feature extraction
- 3) To train a classifier using the pretrained model
- 4) To evaluate the performance of the model

Scene Understanding : Scene understanding refers to identifying and interpreting objects present in an image. In this experiment, the model learns to recognize visual patterns such as shapes and textures and classify images into categories (cat or dog). Using transfer learning, the pretrained model already understands general visual features, which are reused for recognizing new images efficiently.

Dataset Description: The dataset used is the Cats vs Dogs dataset, obtained from TensorFlow Datasets. It contains labeled images of cats and dogs, Used for binary image classification, Images are preprocessed and resized to 224×224. **Dataset Source:** TensorFlow Datasets (tfds.load) **Base Paper:** ImageNet Classification with Deep Convolutional Neural Networks .

Implementation(Code, Results)

Step 1: Import Libraries

```
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications import MobileNet
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.preprocessing import image
```

Step 2: Load and Preprocess Dataset

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 24

```
# Load dataset
dataset, info = tfds.load(
    'cats_vs_dogs',
    split='train',
    with_info=True,
    as_supervised=True
)

# Preprocess
def preprocess(img, label):
    img = tf.image.resize(img, (224,224))
    img = img / 255.0
    return img, label

dataset = dataset.map(preprocess).batch(16)
```

Step 3: Load Pretrained Model

```
base_model = MobileNet(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)
```

Step 4: Freeze Layers

```
for layer in base_model.layers:
    layer.trainable = False
```

Step 5: Build Model

```
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(64, activation='relu'),
    Dense(2, activation='softmax')
])
```

Step 6: Compile and Train Model

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(dataset.take(50), epochs=2)
```

Step 7: Test with Input Image

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 25

```

# Load image
img_path = '/content/cat.jpg' # 🖱️ your image path
img = image.load_img(img_path, target_size=(224,224))
img_array = image.img_to_array(img)

# Preprocess
img_array = img_array / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Predict
prediction = model.predict(img_array)
class_index = np.argmax(prediction)

# Class labels (adjust if needed)
labels = ['Cat', 'Dog']

# Show image with prediction
plt.imshow(img)
plt.title(f'Prediction: {labels[class_index]}")
plt.axis('off')
plt.show()

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 26

Results

Prediction: Dog



Prediction: Cat



Conclusion: Transfer learning provides an efficient approach for image classification tasks. By using a pretrained model like MobileNet, high accuracy was achieved with minimal training data and time. The experiment demonstrates that pretrained models can effectively transfer knowledge to new tasks.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 27

Lab 10: Object Detection using -- Haar Cascades

Date of the Session: ____/____/____

Time: ____

Aim: To implement real-time object detection using a pre-trained Haar Cascade classifier.

Introduction: Object detection is a fundamental computer vision task that involves identifying and locating objects within images or video streams. One of the earliest and most efficient real-time object detection methods is the Haar Cascade Classifier, introduced by Paul Viola and Michael Jones in 2001. The method uses Haar-like features, integral images, AdaBoost learning, and a cascade classifier structure to detect objects quickly and efficiently. It is widely implemented in the OpenCV library for tasks like face detection, eye detection, and smile detection. Haar Cascades are computationally efficient and suitable for real-time applications. Application on Particular Scene : Specific Scene: Real-Time Surveillance Monitoring System. In this experiment, Haar Cascade-based object detection is applied to a CCTV surveillance environment, where a fixed security camera monitors an indoor or semi-outdoor area such as a corridor, entrance gate, or office lobby.

Objectives:

- 1) To understand the working principle of Haar-like features
- 2) To use pre-trained Haar Cascade models
- 3) To perform real-time object detection using a webcam
- 4) To visualize detected objects using bounding boxes
- 5) To evaluate detection performance in a live environment

Scene Description:

- 6) Environment: Office corridor / building entrance
- 7) Camera Type: Fixed CCTV camera
- 8) Lighting: Mixed lighting (natural + artificial)
- 9) Objects of Interest: Human faces entering or passing through the monitored area
- 10) Background: Mostly static with occasional movement

The system continuously processes video frames from the surveillance camera and detects human faces in real time. When a face appears in the monitored region, it is highlighted with a bounding box for security tracking

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 28

Dataset Description: The dataset consists of CCTV surveillance video captured from a fixed-position camera. The video contains real-world scenes with moving objects such as pedestrians and/or vehicles under varying lighting conditions. The footage is stored in standard video format (e.g., .mp4) with typical resolution (640×480 or 1280×720) and processed frame-by-frame. Each frame is converted to grayscale and object detection is performed using the Haar Cascade method implemented in OpenCV.

Implementation :-

```
import cv2
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
video_path = "Abuse001_x264.mp4"
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: Cannot open video file")
    exit()
frame_width = int(cap.get(3))
frame_height = int(cap.get(4))
fps = int(cap.get(cv2.CAP_PROP_FPS))
fourcc = cv2.VideoWriter_fourcc(*'XVID')
out = cv2.VideoWriter('output_human_detected.avi',
                    fourcc, fps, (frame_width, frame_height))
print("Processing Video... Press 'q' to stop")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    # Resize (improves detection speed)
    frame = cv2.resize(frame, (640, 480))
    # Detect humans
    boxes, weights = hog.detectMultiScale(
        frame,
        winStride=(8, 8),
        padding=(8, 8),
        scale=1.05
    )
    # Draw bounding boxes
    for (x, y, w, h) in boxes:
        cv2.rectangle(frame,
            (x, y),
            (x + w, y + h),
            (0, 0, 255), 2)
    cv2.imshow("Human Detection - Surveillance", frame)
    out.write(frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
```

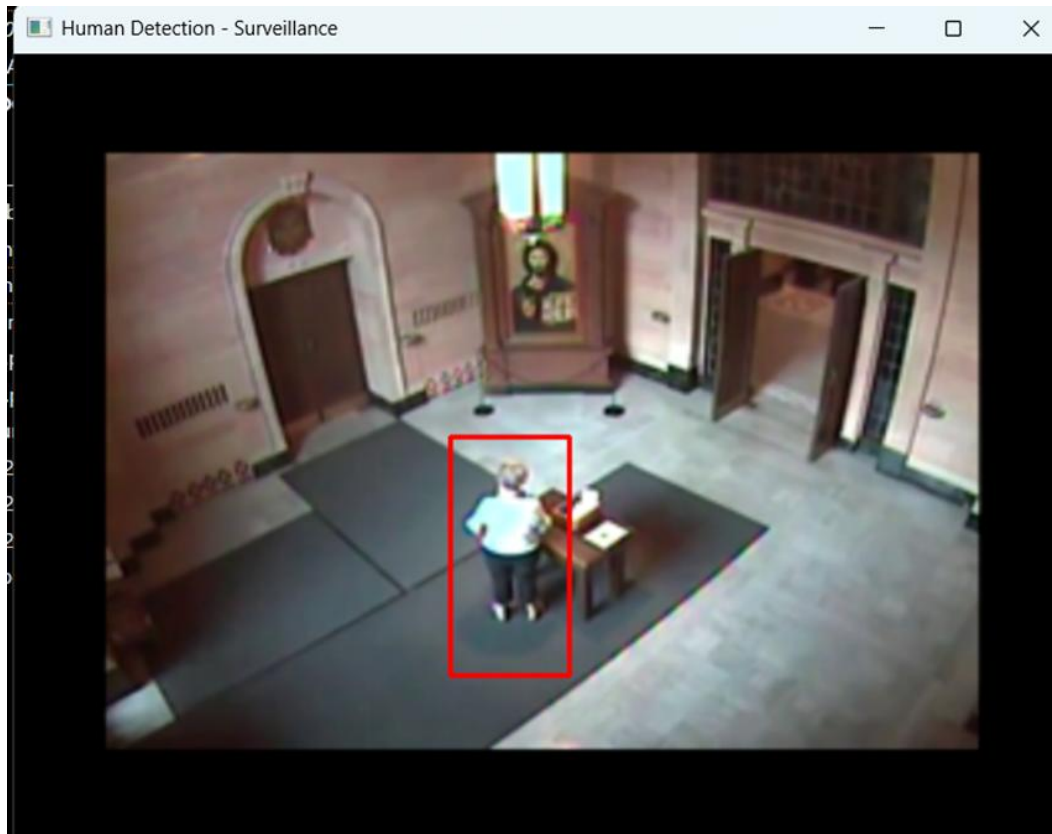
Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 29

```

out.release()
cv2.destroyAllWindows()
print("Done. Output saved as output_human_detected.avi")

```

Result:-



	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 30

Lab 11: Object Detection – YOLO (Pretrained Model)

Date of the Session: ____/____/____

Time: ____

Aim: To perform real-time object detection on an image or video using a pretrained YOLO model.

Introduction: Object detection is a computer vision task that identifies and localizes objects in images or videos using bounding boxes. Modern deep learning models perform detection in real-time with high accuracy. In this experiment, a pretrained YOLO (You Only Look Once) model is used. YOLO is a single-stage detector that predicts bounding boxes and class probabilities in one forward pass, making it suitable for real-time applications. The model is implemented using OpenCV and pretrained on the COCO dataset. . Application / Justification: YOLO-based object detection is widely used in: CCTV surveillance systems, traffic monitoring, autonomous vehicles, smart city applications, security and threat detection. In surveillance scenes, YOLO can detect persons, vehicles, and other objects accurately even under varying lighting conditions, making it more reliable than traditional methods.

Objectives:

- 1) To understand the working of YOLO object detection.
- 2) To load and use a pretrained YOLO model.
- 3) To detect multiple objects in a scene.
- 4) To draw bounding boxes with class labels.
- 5) To analyze detection results.

Dataset Description:

The pretrained YOLO model is trained on the COCO (Common Objects in Context) dataset. Contains 80 object categories (person, car, dog, etc.) Large-scale real-world images, Annotated with bounding boxes, Used for benchmarking object detection models. For this experiment, a sample image or video is used as input, and detection is performed using pretrained weights.

6.Code:-

```
import cv2
from ultralytics import YOLO
import numpy as np
model = YOLO('yolov8n.pt')
# Open video file
cap = cv2.VideoCapture("00065.mp4")
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 31

```

# Get video properties
fps = int(cap.get(cv2.CAP_PROP_FPS))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
while True:
    ret, frame = cap.read()
    if not ret:
        break
    # Run YOLOv8 inference on the frame
    results = model(frame, conf=0.5)
    # Visualize the results
    annotated_frame = results[0].plot()
    # Display the annotated frame
    cv2.imshow("YOLOv8 Detection", annotated_frame)
    # Press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()

```

Output:-



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 32

Lab 12: Object Detection using R-CNN (Region-Based Convolutional Neural Network)

Date of the Session: ____/____/____

Time: ____

AIM: To perform object detection on images using the R-CNN approach and analyze its performance.

Introduction: Object detection is a computer vision task that identifies objects in an image and locates them using bounding boxes. Traditional methods were limited in accuracy, but deep learning improved performance significantly. In this experiment, R-CNN (Region-Based Convolutional Neural Network) is used. R-CNN first generates region proposals and then classifies each region using a Convolutional Neural Network (CNN). It was one of the first deep learning-based object detection models that achieved high accuracy.

Objectives:

- 1) To understand the working of R-CNN.
- 2) To generate region proposals using Selective Search.
- 3) To classify proposed regions using a CNN model.
- 4) To draw bounding boxes around detected objects.
- 5) To evaluate detection results.

Application : R-CNN-based detection is useful in: CCTV surveillance systems, Medical image analysis, Autonomous vehicles, Security monitoring, Traffic management. In surveillance scenes, R-CNN can detect objects such as persons and vehicles with higher accuracy compared to traditional feature-based methods.

Dataset Description: For this experiment, a sample image dataset containing objects such as persons, cars, and everyday objects is used. datasets used for R-CNN training : COCO. These datasets contain: Annotated images, Bounding box coordinates, Multiple object categories, Real-world scenes

6.Code:-

```
import cv2
import numpy as np
from tensorflow.keras.applications.mobilenet_v2 import MobileNetV2, preprocess_input,
decode_predictions
from tensorflow.keras.preprocessing.image import img_to_array

# Load pretrained CNN model
model = MobileNetV2(weights="imagenet")

# Initialize Selective Search
ss = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 33

```

# Open video
cap = cv2.VideoCapture("input.mp4")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    ss.setBaseImage(frame)
    ss.switchToSelectiveSearchFast()
    rects = ss.process()
    proposals = rects[:100] # Limit number for speed
    for (x, y, w, h) in proposals:
        roi = frame[y:y+h, x:x+w]
        if roi.size == 0:
            continue
        try:
            roi_resized = cv2.resize(roi, (224, 224))
        except:
            continue
        roi_array = img_to_array(roi_resized)
        roi_array = np.expand_dims(roi_array, axis=0)
        roi_array = preprocess_input(roi_array)
        preds = model.predict(roi_array, verbose=0)
        label = decode_predictions(preds, top=1)[0][0]
        confidence = label[2]
        if confidence > 0.90:
            text = f'{label[1]}: {confidence:.2f}'
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.putText(frame, text, (x, y-5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                        (0, 255, 0), 2)
    cv2.imshow("R-CNN Video Detection", frame)

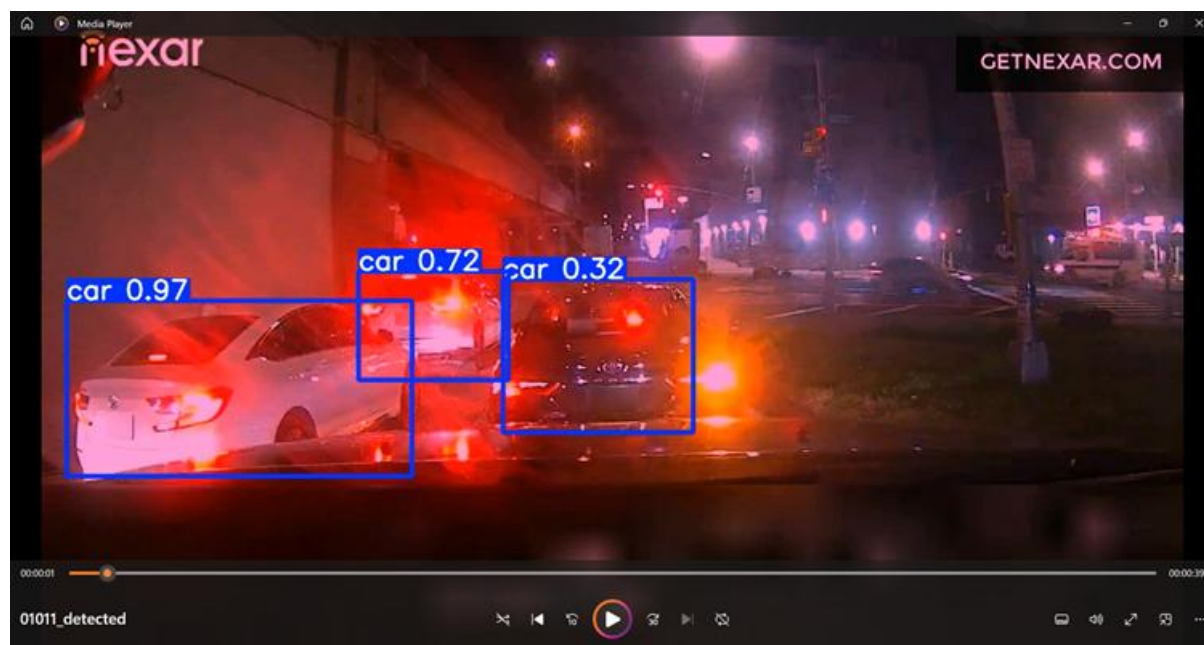
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
cap.release()

cv2.destroyAllWindows()

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 34

Result:-



	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 35

Lab 13: Semantic Segmentation using U-Net

Date of the Session: ____/____/____

Time: ____

Aim: To implement a U-Net model using TensorFlow for semantic segmentation and evaluate its performance on a small scene understanding dataset.

Introduction: Semantic segmentation is a fundamental task in visual recognition and scene understanding where each pixel in an image is classified into a specific category. Unlike image classification (which assigns one label per image), semantic segmentation provides dense predictions, meaning every pixel is labeled. It is widely used in: Autonomous driving (road, pedestrians, vehicles), Medical imaging (tumor segmentation), Satellite imagery (land cover classification), Robotics and smart surveillance

Objectives

- 1) To understand pixel-wise classification.
 - 2) To implement encoder-decoder architecture.
 - 3) To train a U-Net model in Google Colab.
 - 4) To visualize predicted segmentation masks.
- To evaluate segmentation performance.

Scene Understanding : In autonomous driving, understanding the scene is critical. For example: Road → Drivable area, Sky → Ignore, Pedestrian → Avoid collision, Vehicles → Track. Semantic segmentation enables the system to understand *where* objects are, not just *what* they are. This experiment simulates real-world scene understanding.

Dataset Description: We use the CamVid Dataset for scene-level semantic segmentation. Dataset details: 701 labeled urban driving images,, RGB street scene images , Pixel-wise semantic, segmentation masks, 11 semantic classes, Image resolution: 720×960 , Color-coded annotation masks. The dataset contains complex real-world road scenes including vehicles, pedestrians, buildings, roads, trees, and other urban elements. Each image has a corresponding mask where every pixel is assigned to one of the 11 semantic categories.

Implementation

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
import os
import zipfile
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 36


```

print("TensorFlow Version:", tf.__version__)

with zipfile.ZipFile("CamVid.zip", 'r') as zip_ref:
    zip_ref.extractall("CamVid")

IMG_SIZE = 128
BATCH_SIZE = 8
NUM_CLASSES = 11

image_dir = "CamVid/CamVid/train"
mask_dir = "CamVid/CamVid/train_labels"

# 2. CamVid Color Map (11 Classes)

# Standard CamVid color mapping
COLOR_MAP = {
    (128,128,128): 0, # Sky
    (128,0,0): 1, # Building
    (192,192,128): 2, # Pole
    (128,64,128): 3, # Road
    (0,0,192): 4, # Pavement
    (128,128,0): 5, # Tree
    (192,128,128): 6, # SignSymbol
    (64,64,128): 7, # Fence
    (64,0,128): 8, # Car
    (64,64,0): 9, # Pedestrian
    (0,128,192): 10 # Bicyclist
}

def rgb_to_class(mask):
    mask = tf.cast(mask, tf.uint8)
    class_mask = tf.zeros((IMG_SIZE, IMG_SIZE), dtype=tf.int32)

    for color, label in COLOR_MAP.items():
        color_tensor = tf.constant(color, dtype=tf.uint8)
        matches = tf.reduce_all(tf.equal(mask, color_tensor), axis=-1)

        label_tensor = tf.cast(label, tf.int32)
        class_mask = tf.where(matches, label_tensor, class_mask)

    return tf.expand_dims(class_mask, axis=-1)

# 3. Load Images & Masks

def load_image_mask(image_path, mask_path):

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 37

```

image = tf.io.read_file(image_path)
image = tf.image.decode_png(image, channels=3)
image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
image = tf.cast(image, tf.float32) / 255.0

mask = tf.io.read_file(mask_path)
mask = tf.image.decode_png(mask, channels=3)
mask = tf.image.resize(mask, (IMG_SIZE, IMG_SIZE), method="nearest")
mask = rgb_to_class(mask)
mask = tf.cast(mask, tf.int32)

```

```

return image, mask

```

```

image_files = sorted([os.path.join(image_dir, f) for f in os.listdir(image_dir)])
mask_files = sorted([os.path.join(mask_dir, f) for f in os.listdir(mask_dir)])

```

```

dataset = tf.data.Dataset.from_tensor_slices((image_files, mask_files))
dataset = dataset.map(load_image_mask)

```

```

# Split train/val
dataset = dataset.shuffle(500)
train_size = int(0.8 * len(image_files))

```

```

train_dataset = dataset.take(train_size).batch(BATCH_SIZE)
val_dataset = dataset.skip(train_size).batch(BATCH_SIZE)

```

```

# 4. Build U-Net

```

```

def conv_block(inputs, filters):
    x = tf.keras.layers.Conv2D(filters, 3, padding='same', activation='relu')(inputs)
    x = tf.keras.layers.Conv2D(filters, 3, padding='same', activation='relu')(x)
    return x

```

```

def build_unet():
    inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

```

```

    c1 = conv_block(inputs, 32)
    p1 = tf.keras.layers.MaxPooling2D()(c1)

```

```

    c2 = conv_block(p1, 64)
    p2 = tf.keras.layers.MaxPooling2D()(c2)

```

```

    c3 = conv_block(p2, 128)
    p3 = tf.keras.layers.MaxPooling2D()(c3)

```

```

    c4 = conv_block(p3, 256)

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 38

```

u1 = tf.keras.layers.UpSampling2D()(c4)
u1 = tf.keras.layers.Concatenate()([u1, c3])
c5 = conv_block(u1, 128)

u2 = tf.keras.layers.UpSampling2D()(c5)
u2 = tf.keras.layers.Concatenate()([u2, c2])
c6 = conv_block(u2, 64)

u3 = tf.keras.layers.UpSampling2D()(c6)
u3 = tf.keras.layers.Concatenate()([u3, c1])
c7 = conv_block(u3, 32)

outputs = tf.keras.layers.Conv2D(NUM_CLASSES, 1, activation='softmax')(c7)

return tf.keras.Model(inputs, outputs)

```

5. Dice Metric

```

def dice_coefficient(y_true, y_pred, smooth=1e-6):
    y_true = tf.squeeze(y_true, axis=-1)
    y_true = tf.one_hot(tf.cast(y_true, tf.int32), depth=NUM_CLASSES)
    y_true = tf.cast(y_true, tf.float32)

    y_pred = tf.nn.softmax(y_pred)

    y_true = tf.reshape(y_true, [-1, NUM_CLASSES])
    y_pred = tf.reshape(y_pred, [-1, NUM_CLASSES])

    intersection = tf.reduce_sum(y_true * y_pred, axis=0)
    denominator = tf.reduce_sum(y_true + y_pred, axis=0)

    dice = (2. * intersection + smooth) / (denominator + smooth)
    return tf.reduce_mean(dice)

```

```
model = build_unet()
```

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy', dice_coefficient]
)

```

```
model.summary()
```

6. Train

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 39

```
history = model.fit(train_dataset,
                    validation_data=val_dataset,
                    epochs=50)
```

7. Visualization

```
for images, masks in val_dataset.take(1):
    preds = model.predict(images)
    preds = tf.argmax(preds, axis=-1)
```

```
plt.figure(figsize=(12,4))
```

```
plt.subplot(1,3,1)
plt.title("Input")
plt.imshow(images[0])
plt.axis("off")
```

```
plt.subplot(1,3,2)
plt.title("Ground Truth")
plt.imshow(tf.squeeze(masks[0]))
plt.axis("off")
```

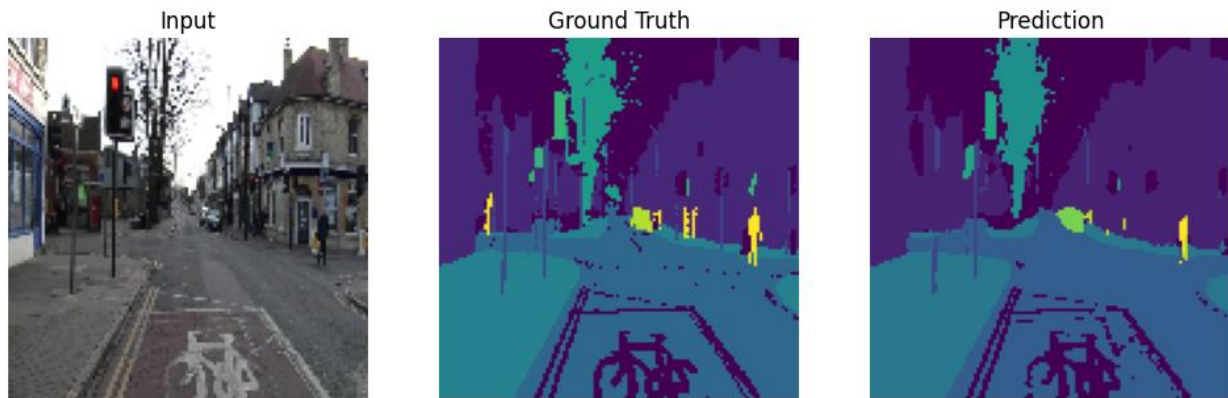
```
plt.subplot(1,3,3)
plt.title("Prediction")
plt.imshow(preds[0])
plt.axis("off")
```

```
plt.show()
```

```
print("CamVid Scene Segmentation Completed Successfully!")
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 40

Results



Conclusion: This experiment successfully implemented the U-Net model using TensorFlow to perform pixel-wise classification for semantic segmentation tasks. It provided a clear understanding of the encoder-decoder architecture and demonstrated how segmentation outputs can be visualized effectively. The experiment also highlighted the practical application of scene understanding concepts. Overall, semantic segmentation is a vital component of advanced AI systems, with significant applications in autonomous vehicles, medical diagnosis, and robotics.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 41

Lab 15: Scene Classification

Date of the Session: ____/____/____

Time: _____

Aim: To implement a deep learning-based approach for classifying images into different scene categories using a pre-trained convolutional neural network.

Introduction: Scene classification is a fundamental problem in the field of computer vision and visual recognition, where the goal is to assign a semantic label to an entire image based on its overall content. Unlike object detection, which focuses on identifying individual objects, scene classification captures the global context of the image such as indoor, outdoor, urban, or natural environments. With the advancement of deep learning, especially Convolutional Neural Networks (CNNs), scene classification has become highly accurate and efficient. Pre-trained models like ResNet, VGG, and MobileNet are widely used as they are trained on large datasets and can generalize well to new images. This experiment demonstrates how a pre-trained CNN model can be used to classify scenes effectively.

Objectives:

- 1) To understand the concept of scene-level image classification
- 2) To explore the use of pre-trained deep learning models
- 3) To preprocess images for model input
- 4) To perform inference using a trained model
- 5) To interpret classification results

Scene Understanding: Scene classification plays a crucial role in enabling machines to understand their surroundings at a higher level. It is widely used in: Autonomous driving systems to distinguish between highways, city streets, and rural roads, Smart photo organization systems to automatically group images, Surveillance systems for environment monitoring, Robotics for navigation and decision-making. By recognizing the scene context, systems can make more intelligent and context-aware decisions.

Dataset Description: The dataset used in this experiment is the ImageNet Dataset, which is a large-scale visual database designed for image recognition tasks. It contains over 1 million images categorized into 1000 classes. Classes include objects and scenes such as *cat, dog, car, forest, building*, etc. Each image is labeled with a corresponding category. The pretrained model (ResNet-18) used in this experiment has been trained on this dataset. The dataset enables the model to learn rich visual features and generalize well to new input images.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 1

Code

```
import torch
from torchvision import models, transforms
from PIL import Image
import matplotlib.pyplot as plt
import urllib.request

classes = urllib.request.urlopen(
    "https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt"
).read().decode("utf-8").split("\n")

# =====
# LOAD PRETRAINED MODEL
# =====
model = models.resnet18(pretrained=True)
model.eval()

# =====
# IMAGE TRANSFORM
# =====
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

# =====
# SCENE CLASSIFICATION FUNCTION
# =====
def classify_scene(label):
    label = label.lower()

    # 🌿 Nature scenes
    if any(x in label for x in ["mountain", "valley", "volcano", "alp"]):
        return "Mountain Scene"
    elif any(x in label for x in ["forest", "tree", "wood"]):
        return "Forest Scene"
    elif any(x in label for x in ["sea", "ocean", "beach", "sandbar", "coast"]):
        return "Beach Scene"

    # 🏠 Indoor scenes
    elif any(x in label for x in ["bedroom", "bed"]):
        return "Bedroom Scene"
    elif any(x in label for x in ["kitchen", "dining", "table"]):
        return "Indoor Scene"
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 2

```

# 🏙️ Urban scenes
elif any(x in label for x in ["building", "street", "car", "cab", "traffic"]):
    return "Urban Scene"

# 🌳 Natural life
elif any(x in label for x in ["bird", "animal"]):
    return "Nature Scene"

else:
    return "Unknown Scene"

```

```

image_files = ["/content/beach.jpg"] # change path if needed

```

```

for img_path in image_files:
    try:
        # Load image
        img = Image.open(img_path).convert("RGB")

        # Transform image
        img_t = transform(img).unsqueeze(0)

        # Prediction
        with torch.no_grad():
            output = model(img_t)
            _, pred = torch.max(output, 1)

        # Labels
        original_label = classes[pred.item()]
        scene_label = classify_scene(original_label)

        # Display image
        plt.imshow(img)
        plt.title(f"Scene: {scene_label}")
        plt.axis("off")
        plt.show()

        # Print results
        print("Original Label:", original_label)
        print("Scene Classification:", scene_label)
        print("-" * 40)

```

```

except Exception as e:
    print(f"Error loading image {img_path}: {e}")

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 3

Output

Scene: Mountain Scene



Scene: Beach Scene



Original Label: valley

Original Label: seashore

Conclusion: In this experiment, scene classification was performed using a pretrained Convolutional Neural Network, ResNet-18. The model was able to analyze the input image and successfully predict its corresponding class label based on features learned from the ImageNet Dataset. The experiment demonstrates that pretrained CNN models can effectively extract high-level features such as textures, shapes, and patterns, enabling accurate classification of images without the need for extensive training. It also highlights the efficiency and practicality of transfer learning in solving real-world visual recognition problems. Thus, scene classification using pretrained deep learning models proves to be a powerful and reliable approach for understanding and categorizing visual data.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 4

Lab 16: Scene Graph Generation

Date of the Session: ____/____/____

Time: _____

Aim: To generate a scene graph from an input image by detecting objects and identifying the relationships between them using deep learning techniques.

Introduction : Scene Graph Generation is an advanced task in computer vision that aims to provide a detailed and structured understanding of visual scenes. Unlike traditional image classification, which assigns a single label to an image, scene graph generation identifies multiple objects within an image and captures the relationships between them. A scene graph represents an image in the form of a graph, where objects are treated as nodes and the relationships between objects are represented as edges. This structured representation helps in understanding not only *what objects are present* but also *how they interact with each other*. Scene graph generation is widely used in applications such as image captioning, visual question answering, robotics, and autonomous systems. It bridges the gap between visual data and natural language by enabling machines to interpret complex scenes more intelligently.

Objectives

- 1) To detect multiple objects present in an image
- 2) To identify relationships between the detected objects
- 3) To represent the image in the form of a structured graph
- 4) To understand interactions between objects in a scene
- 5) To enhance scene understanding beyond basic classification

Scene Understanding : Scene Graph Generation provides a deeper level of image understanding by capturing both objects and their relationships. Instead of simply identifying what is present in an image, it explains how different objects are connected and interact with each other. For example, in an image containing a person and a bicycle, the model not only detects the objects but also identifies the relationship such as “person riding bicycle.” This structured representation helps in better interpretation of complex scenes and is useful in applications like image captioning, visual question answering, and robotics.

Dataset Description: The model was trained and evaluated using the COCO dataset, which contains approximately 330,000 images. This large-scale dataset includes 80 different object categories, enabling robust object detection and recognition across a wide range of everyday items. Some of the categories include people, cars, dogs, chairs, and bottles.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 5

Code

```
import torch
import torchvision
from torchvision import transforms
import cv2
import matplotlib.pyplot as plt
img_path = "/content/img2-sgg.png" # change your image path
image_bgr = cv2.imread(img_path)
if image_bgr is None:
    print("Image not found")
    exit()

image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()
COCO_CLASSES = [
    '__background__', 'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign',
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella', 'N/A',
    'N/A', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard',
    'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard',
    'surfboard', 'tennis racket', 'bottle', 'N/A', 'wine glass', 'cup', 'fork',
    'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli',
    'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
    'potted plant', 'bed', 'N/A', 'dining table', 'N/A', 'N/A', 'toilet', 'N/A',
    'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave',
    'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book', 'clock', 'vase',
    'scissors', 'teddy bear', 'hair drier', 'toothbrush'
]

transform = transforms.Compose([
    transforms.ToTensor()
])
img_tensor = transform(image)
with torch.no_grad():
    output = model([img_tensor])[0]
boxes = output["boxes"].cpu().numpy()
labels = output["labels"].cpu().numpy()
scores = output["scores"].cpu().numpy()
threshold = 0.7
nodes = []
class_counter = {}
for obj in objects: # objects you already created
    base = obj["base_label"]
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 6

```

if base not in class_counter:
    class_counter[base] = 0
else:
    class_counter[base] += 1
node_name = base.capitalize() + "_" + str(class_counter[base])
nodes.append({
    "name": node_name,
    "base": base,
    "box": obj["box"]
})
def box_center(box):
    x1, y1, x2, y2 = box
    return ((x1+x2)/2, (y1+y2)/2)
def box_area(box):
    x1, y1, x2, y2 = box
    return abs(x2-x1) * abs(y2-y1)
def iou(boxA, boxB):
    ax1, ay1, ax2, ay2 = boxA
    bx1, by1, bx2, by2 = boxB
    ix1 = max(ax1, bx1)
    iy1 = max(ay1, by1)
    ix2 = min(ax2, bx2)
    iy2 = min(ay2, by2)
    iw = max(0, ix2 - ix1)
    ih = max(0, iy2 - iy1)
    inter = iw * ih
    areaA = box_area(boxA)
    areaB = box_area(boxB)
    union = areaA + areaB - inter
    if union == 0:
        return 0
    return inter / union

draw_img = image.copy()
for obj in objects:
    x1, y1, x2, y2 = map(int, obj["box"])
    cv2.rectangle(draw_img, (x1,y1), (x2,y2), (0,255,0), 2)
    cv2.putText(draw_img, obj["label"], (x1, y1-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,0,0), 2)
plt.figure(figsize=(8,8))
plt.imshow(draw_img)
plt.axis("off")
plt.show()
def get_relation(boxA, boxB):
    ax1, ay1, ax2, ay2 = boxA
    bx1, by1, bx2, by2 = boxB

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 7

```

    acx = (ax1 + ax2) / 2.0
    acy = (ay1 + ay2) / 2.0
    bcx = (bx1 + bx2) / 2.0
    bcy = (by1 + by2) / 2.0
    # horizontal relation
    if acx < bcx:
        horiz = "left of"
    else:
        horiz = "right of"

    # vertical relation
    if acy < bcy:
        vert = "above"
    else:
        vert = "below"
    return horiz, vert
scene_graph = []
for i in range(len(objects)):
    for j in range(len(objects)):
        if i == j:
            continue
        obj1 = objects[i]["label"]
        box1 = objects[i]["box"]
        obj2 = objects[j]["label"]
        box2 = objects[j]["box"]
        h, v = get_relation(box1, box2)
        scene_graph.append((obj1, h, obj2))
        scene_graph.append((obj1, v, obj2))

print("\nScene Graph Triplets (subject - relation - object)\n")
for triplet in scene_graph:
    print(triplet[0], triplet[1], triplet[2])

```

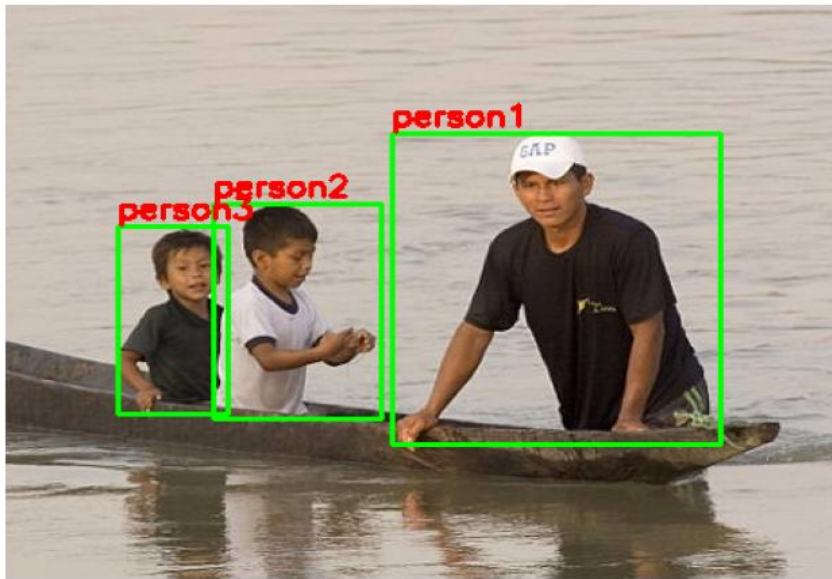
Output: Scene Graph Triplets (subject - relation - object)

```

person1 right of person2
person1 above person2
person1 right of person3
person1 above person3
person2 left of person1
person2 below person1
person2 right of person3
person2 above person3
person3 left of person1
person3 below person1
person3 left of person2
person3 below person2

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 8



Conclusion: In this experiment, scene graph generation was successfully implemented using a pretrained object detection model, Faster R-CNN. The model was able to detect multiple objects in the input image using knowledge learned from the COCO Dataset. Further, spatial relationships such as *left of*, *right of*, *above*, and *below* were computed between detected objects to form meaningful triplets in the form of (*subject – relation – object*). This allowed the image to be represented as a structured scene graph. The experiment demonstrates that combining object detection with spatial reasoning provides a deeper understanding of visual scenes compared to simple classification. Thus, scene graph generation proves to be an effective approach for advanced scene understanding and has applications in areas like image captioning, robotics, and visual reasoning.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 9

Lab 18: 3D Reconstruction Basics

Date of the Session: ____/____/____

Time: _____

Aim: To understand and implement the basic concept of 3D reconstruction using feature detection and matching between multiple images.

Introduction: 3D Reconstruction is a computer vision technique used to generate a three-dimensional representation of a scene from two-dimensional images. It helps in understanding the depth, structure, and spatial arrangement of objects in the real world. The process involves capturing multiple images of the same scene from different viewpoints and identifying corresponding points between them. Using these correspondences, the system estimates depth and reconstructs the 3D structure of the scene. Techniques such as stereo vision and feature matching are commonly used in 3D reconstruction. Feature detection algorithms identify key points in images, and matching these points across images helps in estimating depth and camera motion. This technique is widely used in applications such as robotics, virtual reality, medical imaging, and autonomous navigation. In this experiment, a basic implementation is performed using feature detection and matching to demonstrate the concept of 3D reconstruction.

Objectives:

- 1) To generate a multi-view image dataset by extracting frames from a short video sequence.
- 2) To preprocess the dataset by removing the background from each image, thereby isolating the main object of interest.
- 3) To understand the importance of image overlap and viewpoint variation in 3D reconstruction.
- 4) To detect and match feature points across multiple images using techniques such as ORB (Oriented FAST and Rotated BRIEF).
- 5) To estimate camera positions and orientations using Structure-from-Motion (SfM) methods.
- 6) To reconstruct a sparse and dense 3D model of the object from 2D images.
- 7) To analyze how preprocessing (like background removal) affects the quality and accuracy of the reconstructed 3D model.

Scene Understanding : 3D reconstruction provides a deeper understanding of a scene by estimating depth and spatial relationships between objects. Unlike 2D image analysis, it helps determine how far objects are and how they are positioned in space.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 10

Dataset Description: The dataset used in this experiment was created from a short video sequence captured using a camera. Instead of directly using images, a few seconds of video was recorded focusing on the object/scene of interest. This video was then processed to extract multiple frames at regular intervals, which serve as the input images for the 3D reconstruction pipeline. After frame extraction, background removal techniques were applied to each image to isolate the main object. This preprocessing step helps in reducing noise and unwanted features, allowing the reconstruction algorithm to focus only on the relevant object details. The final dataset consists of a set of cleaned images with minimal background, stored in PNG/JPG format. These images represent different viewpoints of the same object, ensuring sufficient variation for feature detection and matching. The extracted images maintain good overlap between consecutive frames, which is essential for accurate feature matching using algorithms like ORB (Oriented FAST and Rotated BRIEF).

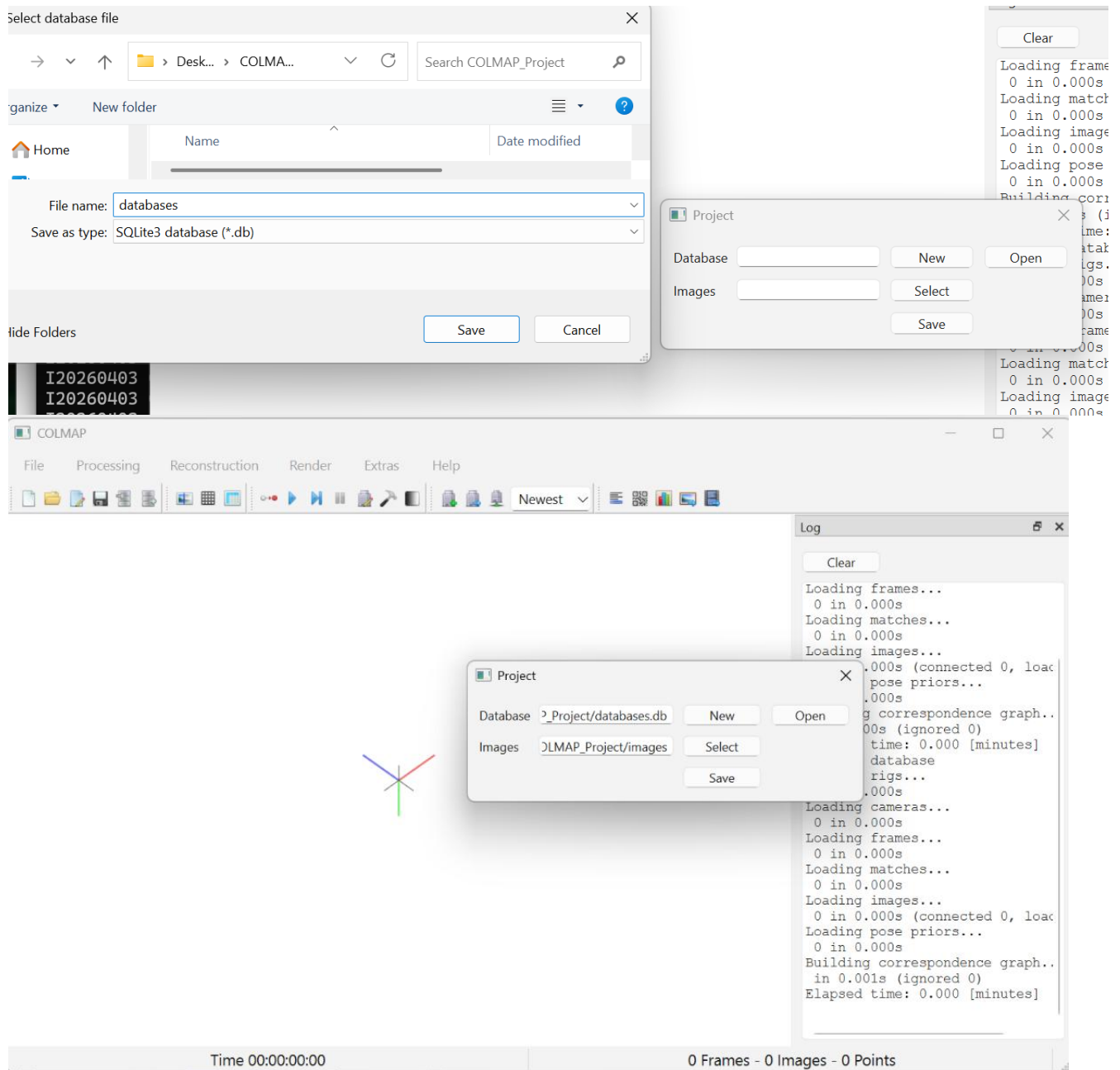
Code: Steps

A short video of the object/scene was captured using a camera to obtain multiple viewpoints.

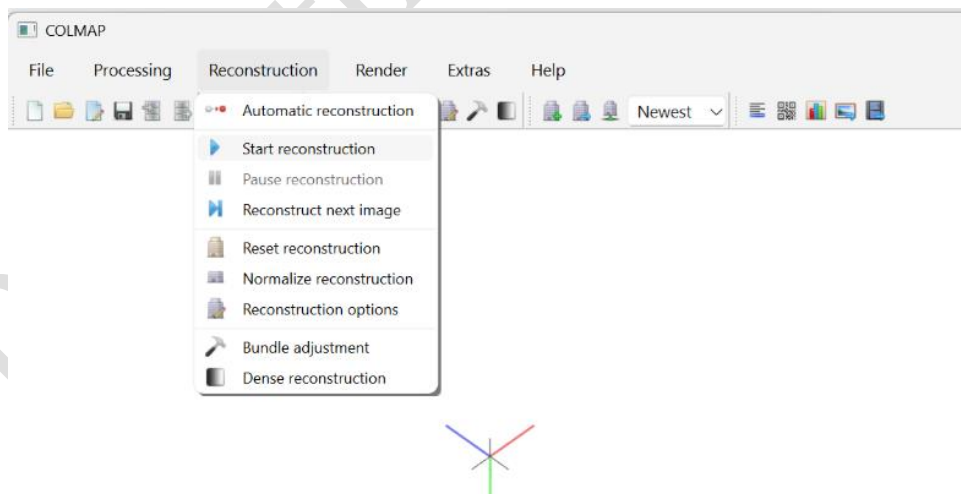
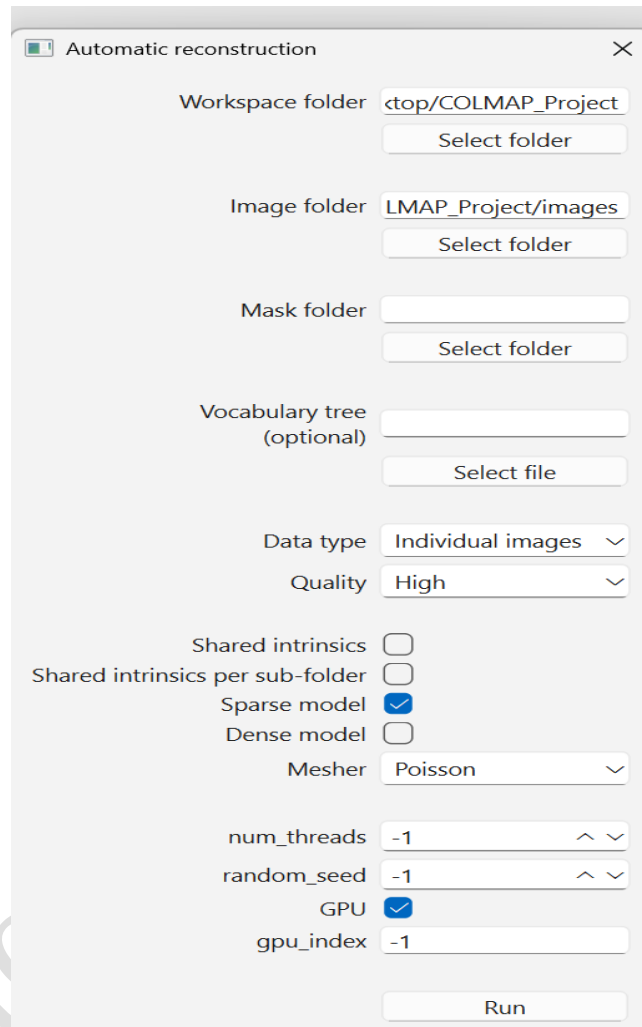
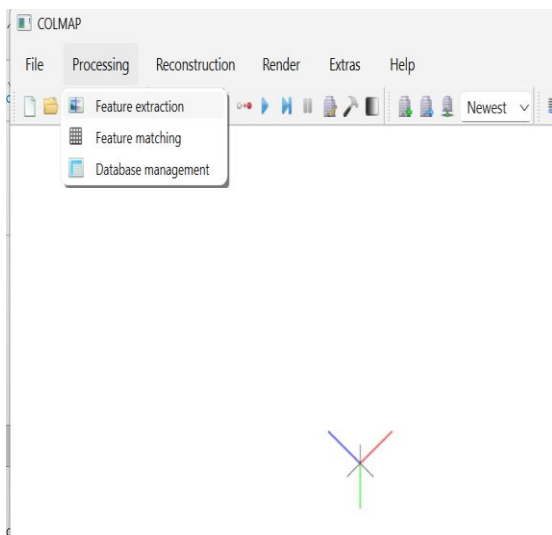
- The recorded video was processed to extract individual frames at regular intervals, generating a set of images.
- Each extracted image was pre-processed by removing the background, and a black background was applied to highlight the main object clearly.
- The processed images were organized into a folder to be used as the input dataset.
- The dataset folder was uploaded to Google Colab for further processing.
- In Colab, the reconstruction environment (COLMAP setup) was initialized, and the input image folder along with a database file was selected.
- Feature extraction was performed on all images (preprocessing step), where key points were detected using feature detection algorithms.
- Feature matching was carried out to find correspondences between images taken from different viewpoints.
- The Structure-from-Motion (SfM) process was executed to estimate camera positions and generate a sparse 3D point cloud.
- Further reconstruction steps were performed to improve the model and obtain better visualization of viewpoints.
- The final reconstructed 3D model was exported and downloaded in .PLY file format.
- The downloaded .ply file was opened in MeshLab for visualization.
 - A clear and detailed 3D reconstruction of the object was successfully obtained and analyzed.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 11

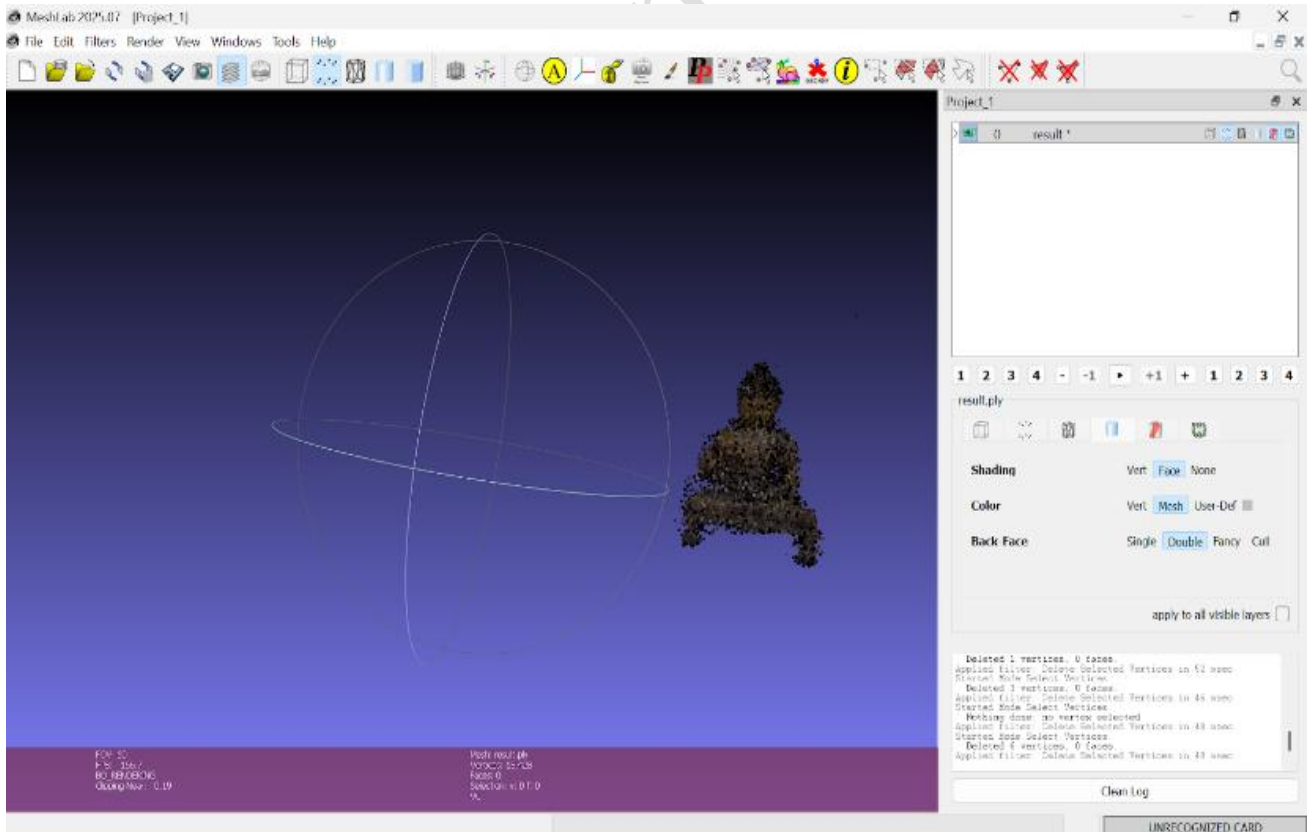
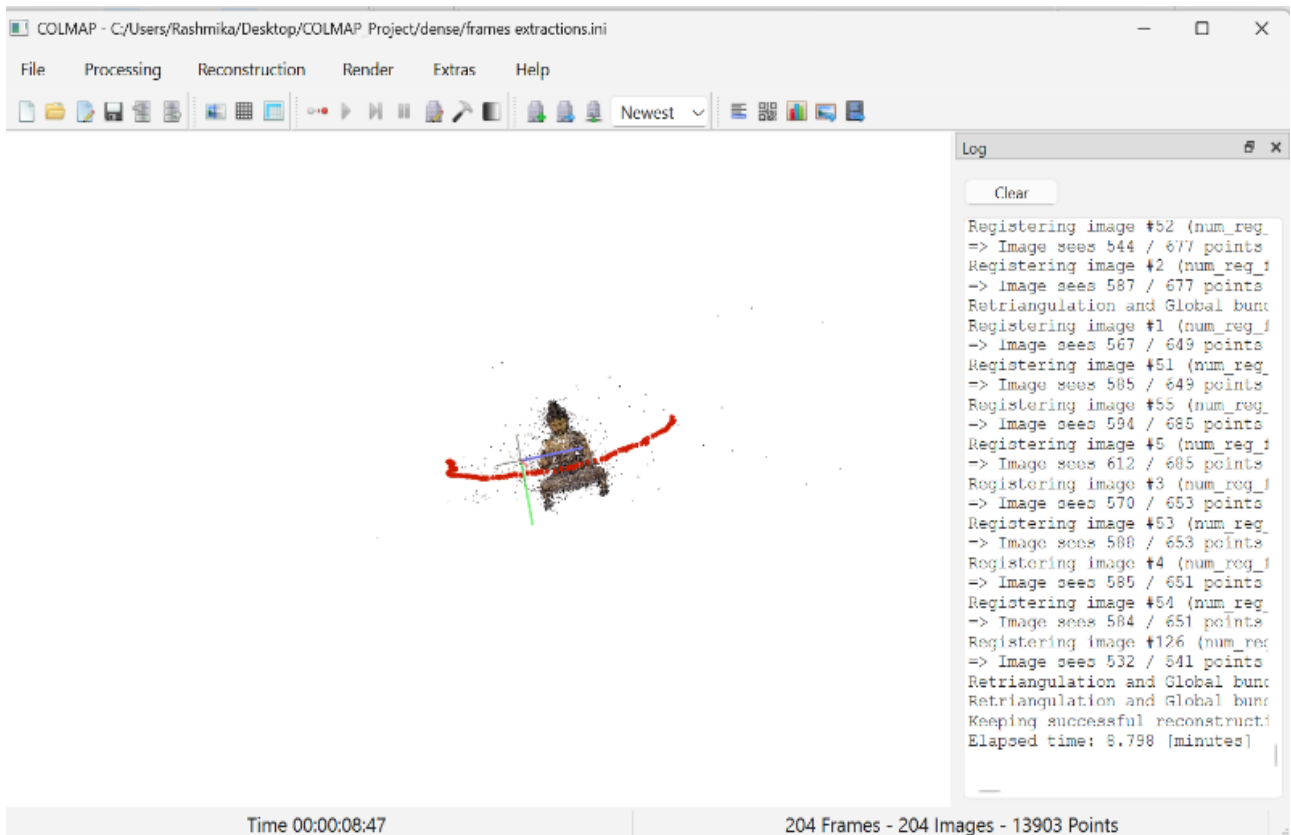
Output



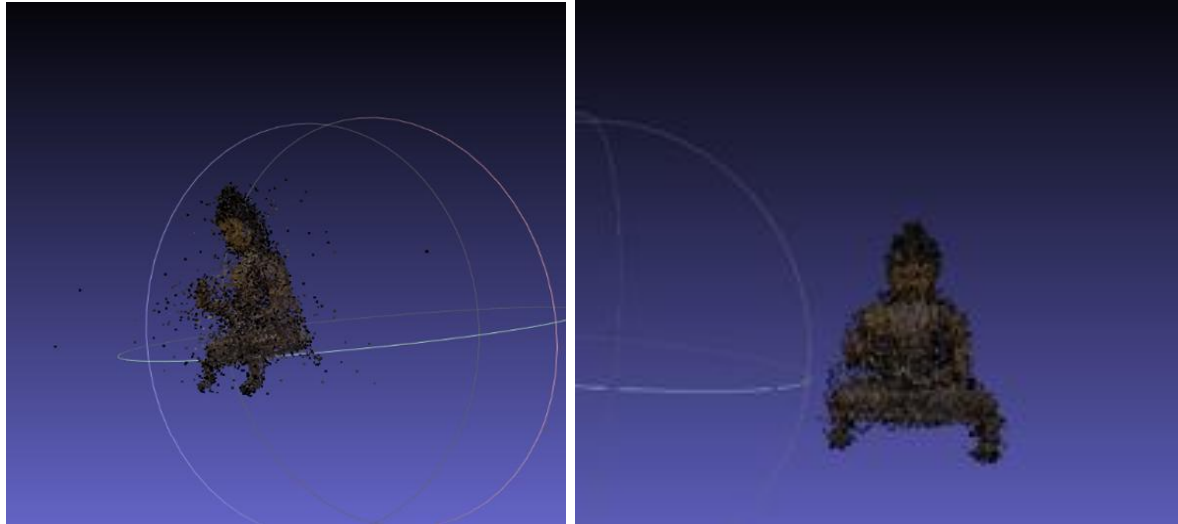
Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 12



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 13



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 14



Conclusion: In this experiment, a 3D reconstruction of an object was successfully achieved using a video-based dataset. A short video was captured and converted into multiple images, which were then pre-processed by removing the background and applying a black background to improve clarity. These processed images were used as input for the reconstruction pipeline. Using COLMAP in Google Colab, feature extraction and matching were performed effectively, followed by Structure-from-Motion to estimate camera positions and generate a 3D point cloud. The final model was exported as a .ply file and visualized in MeshLab, resulting in a clear and accurate 3D representation of the object. This experiment demonstrates that even a simple video can be used to create a reliable multi-view dataset for 3D reconstruction. It also highlights the importance of preprocessing steps, such as background removal and maintaining good image overlap, in improving the overall quality of the reconstructed model.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 15

Lab 19: Robotics Vision – SLAM & Object Grasping

Date of the Session: ____/____/____

Time: ____

Aim: To implement a real-time robotics vision system that performs SLAM and object detection simultaneously on a video stream.

Introduction: Robotics Vision enables robots to perceive, understand, and interact with their surroundings. In this experiment, both: SLAM (Simultaneous Localization and Mapping) → for tracking motion and environment structure Object Detection → for identifying graspable objects are applied simultaneously on a video stream. This mimics real-world robotic systems where: The robot moves and maps the environment (SLAM), While also detecting objects to interact with (grasping).

Objectives

1. Capture video input from camera/video file
2. Perform feature-based SLAM using ORB
3. Track motion between consecutive frames
4. Detect objects using Faster R-CNN
5. Overlay SLAM features and object bounding boxes
6. Visualize combined robotic perception

Application on Particular Scene : This integrated system is used in: Autonomous navigation robots, Warehouse picking robots, Self-driving cars, Surveillance robots, Industrial robotic arms.

Dataset Description: Input: Live webcam OR pre-recorded video. Model Used: Faster R-CNN (pre-trained on COCO dataset), Detects objects like: Bottle, Cup, Chair, Person.

Implementation (Code + Results)

```
import cv2
import numpy as np
import torch
import torchvision
from torchvision import transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 16

```

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.to(device)
model.eval()

```

```

# COCO Labels (simplified)
COCO_CLASSES = [
    '_background_', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',
    'bus', 'train', 'truck', 'boat', 'traffic light'
]
orb = cv2.ORB_create(nfeatures=1000)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

```

```

prev_kp, prev_des = None, None

```

```

cap = cv2.VideoCapture("video.mp4") # or 0 for webcam

```

```

transform = transforms.Compose([transforms.ToTensor()])

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

```

```

# Resize for speed
frame = cv2.resize(frame, (640, 480))
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

kp, des = orb.detectAndCompute(gray, None)

```

```

if prev_des is not None and des is not None:
    matches = bf.match(prev_des, des)
    matches = sorted(matches, key=lambda x: x.distance)

```

```

for m in matches[:30]:
    pt1 = tuple(map(int, prev_kp[m.queryIdx].pt))
    pt2 = tuple(map(int, kp[m.trainIdx].pt))
    cv2.line(frame, pt1, pt2, (0,255,0), 1)

```

```

prev_kp, prev_des = kp, des

```

```

rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
tensor = transform(rgb).to(device)

```

```

with torch.no_grad():
    predictions = model([tensor])

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 17

```

boxes = predictions[0]['boxes']
labels = predictions[0]['labels']
scores = predictions[0]['scores']

threshold = 0.6

for box, label, score in zip(boxes, labels, scores):
    if score > threshold:
        x1, y1, x2, y2 = box.detach().cpu().numpy().astype(int)

        # Draw bounding box (Object Grasp Region)
        cv2.rectangle(frame, (x1,y1), (x2,y2), (255,0,0), 2)

        label_text = COCO_CLASSES[label.item()] if label.item() < len(COCO_CLASSES) else "obj"

        cv2.putText(frame,
                    f'{label_text}: {score:.2f}',
                    (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.5,
                    (255,0,0),
                    2)

cv2.imshow("SLAM + Object Detection", frame)

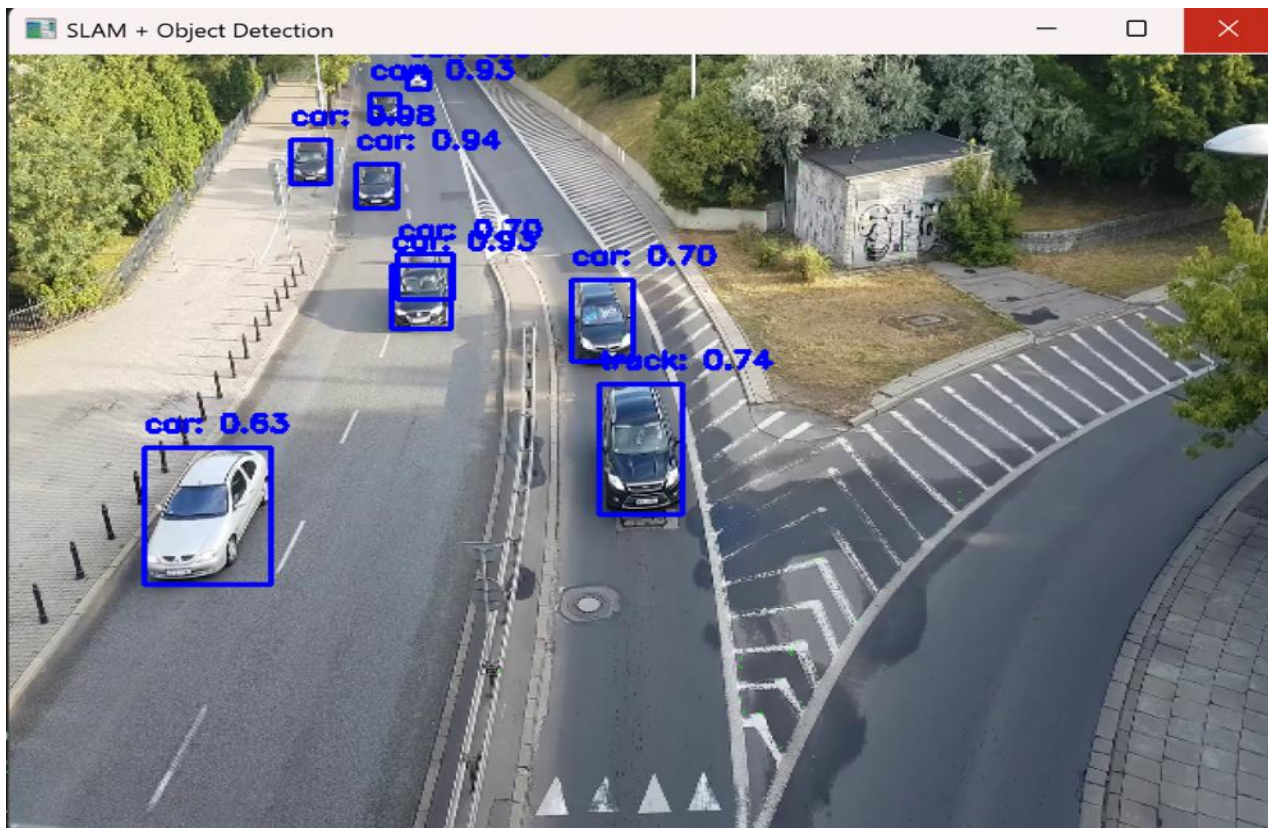
if cv2.waitKey(1) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

```

Output: SLAM Output: a) Feature points tracked across frames, b) Motion represented using green lines. **Object Detection Output:** a) Objects detected with bounding boxes, b) Labels + confidence scores displayed. **Combined Output:** Robot simultaneously: Tracks environment (SLAM), Detects graspable objects

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 18



Conclusion: The system successfully integrates: Real-time SLAM for motion understanding, Object detection for identifying grasp targets. This demonstrates a complete robotics perception pipeline, where a robot can: Navigate its environment, Detect objects, Prepare for grasping actions.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 19